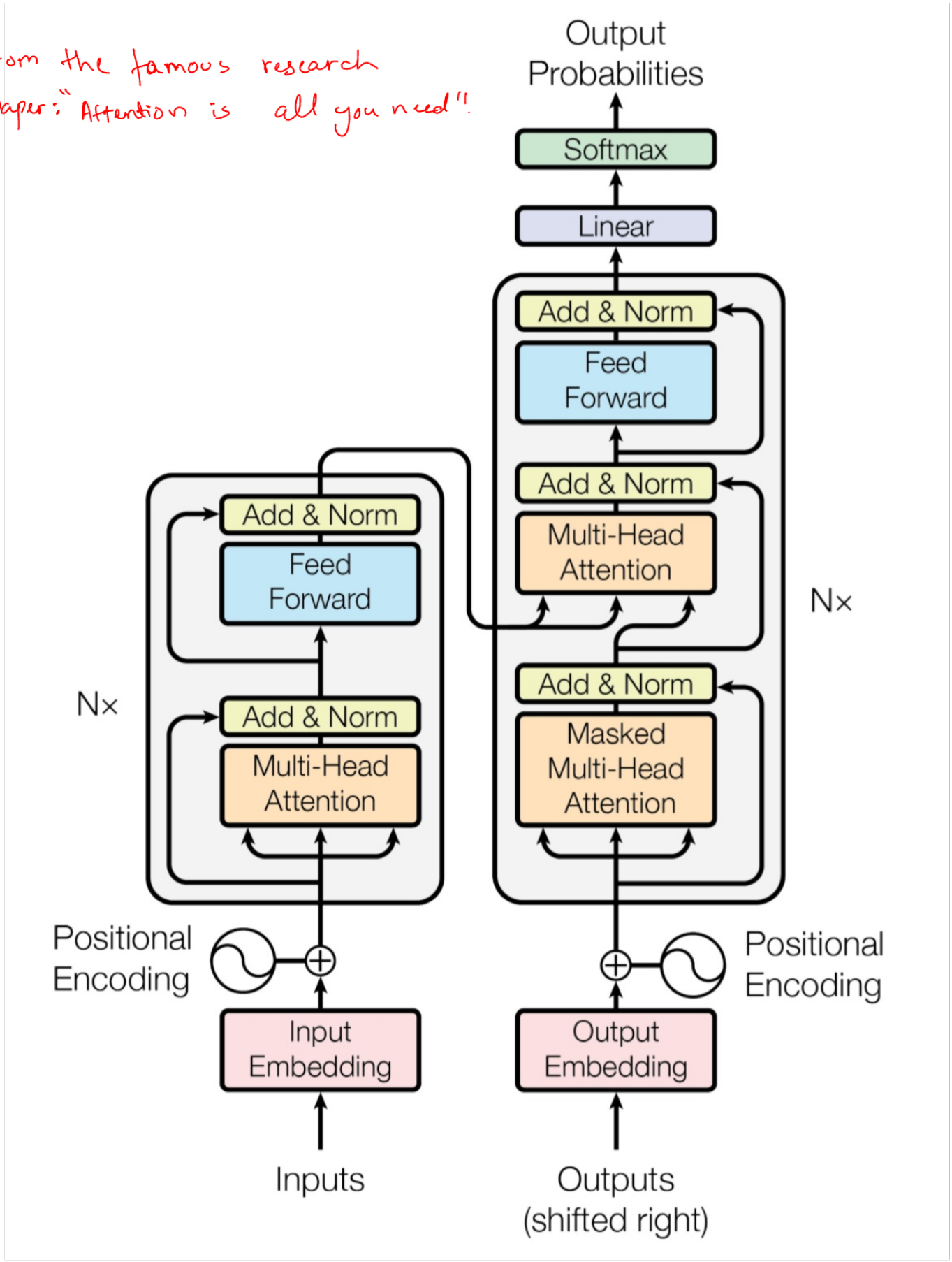
Transformers



Transformers

Transformers are neural network architecture build around self attention that replaced RNN's/CNN's as the dominant way to handle sequences like text.

from the famous research paper: "Attention is all you need".



Self Attention

Self attention is a mechanism that allows each element in a sequence to compute a weighted representation of itself by attending to all other elements in the "same" sequence.

lets first understand why do we need this?

The main problem that we have is "context".

lets take an example :

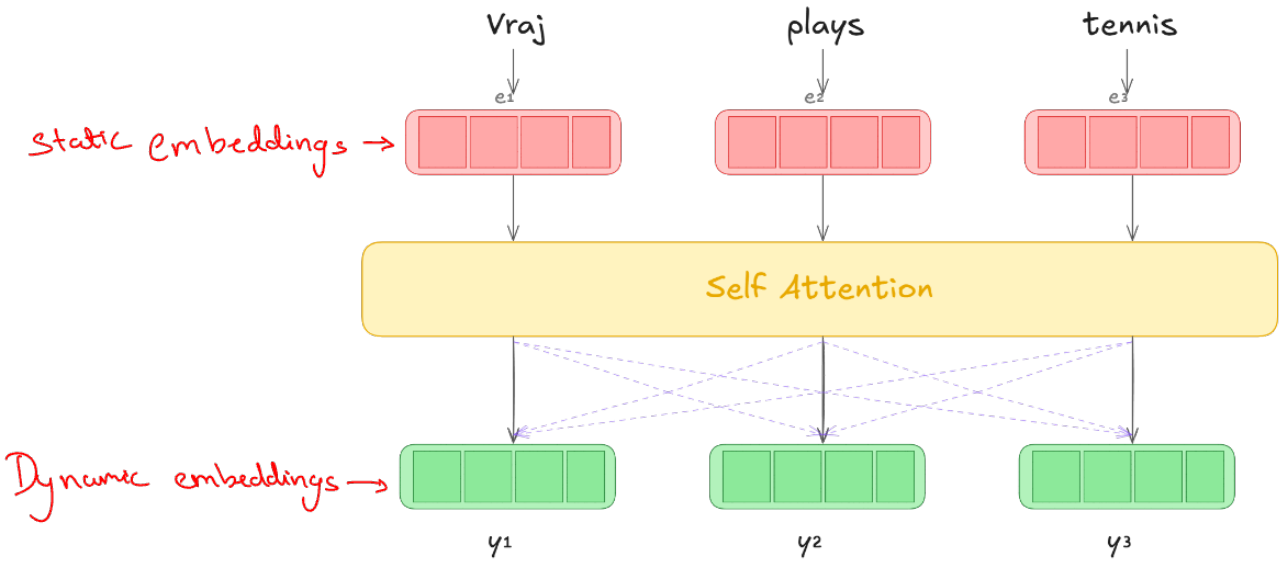
Sentence ① "Bank Money grows"

Sentence ② "River bank flows"

Here, The word "Bank" is used in two different Context, which means a word (embedding) cannot have the same value universally because it depends on the Context.

Because of this we have need dynamic embeddings that are dependend on the context.

So, instead of static embeddings, we can have dynamic embeddings.



lets take the two sentences as examples

1. The train arrives soon.
2. They train models daily.

Now we can consider the word (embedding) "train" has different embeddings depending on the context.

We can say, train is made up of x percent of "The", y % of train, z% arrives and a % of soon.

↓

$$\text{The} = 0.2\,\text{The} + 0.3\,\text{train} + 0.4\,\text{arrives} + 0.1\,\text{soon}$$

$$\text{train} = 0.1\,\text{The} + 0.5\,\text{train} + 0.35\,\text{arrives} + 0.05\,\text{soon}$$

$$\text{arrives} = 0.3\,\text{The} + 0.4\,\text{train} + 0.1\,\text{arrives} + 0.2\,\text{soon}$$

$$\text{soon} = 0.1\,\text{The} + 0.3\,\text{train} + 0.5\,\text{arrives} + 0.1\,\text{soon}$$

Similarly,

$$\text{They} = 0.3\,\text{They} + 0.1\,\text{train} + 0.2\,\text{models} + 0.4\,\text{daily}$$

$$\text{train} = 0.1\,\text{They} + 0.05\,\text{train} + 0.5\,\text{models} + 0.35\,\text{daily}$$

$$\text{models} = 0.4\,\text{They} + 0.1\,\text{train} + 0.4\,\text{models} + 0.1\,\text{daily}$$

$$\text{daily} = 0.3\,\text{They} + 0.4\,\text{train} + 0.1\,\text{models} + 0.2\,\text{daily}$$

Now, lets write the words in embeddings form since we need to convert to embeddings (static here) firstly.

e = embedding

$$e^{new}_{the} = 0.2e_{the} + 0.3e_{train} + 0.4e_{arrives} + 0.1e_{soon}$$

$$e^{new}_{train} = 0.1e_{the} + 0.5e_{train} + 0.35e_{arrives} + 0.05e_{soon}$$

$$e^{new}_{arrives} = 0.3e_{the} + 0.4e_{train} + 0.1e_{arrives} + 0.2e_{soon}$$

$$e^{new}_{soon} = 0.1e_{the} + 0.3e_{train} + 0.5e_{arrives} + 0.1e_{soon}$$

$$e^{new}_{they} = 0.3e_{they} + 0.1e_{train} + 0.2e_{models} + 0.4e_{daily}$$

$$e^{new}_{train} = 0.1e_{they} + 0.05e_{train} + 0.5e_{models} + 0.35e_{daily}$$

$$e^{new}_{models} = 0.4e_{they} + 0.1e_{train} + 0.4e_{models} + 0.1e_{daily}$$

$$e^{new}_{daily} = 0.3e_{they} + 0.4e_{train} + 0.1e_{models} + 0.2e_{daily}$$

The above equations simply shows the similarity between each other.

ex: $e^{new}_{arrives} = 0.3e_{the} + 0.4e_{train} + 0.1e_{arrives} + 0.2e_{soon}$

- $0.3e_{the}$: Similarity between "the" & "arrives"
- $0.4e_{train}$: Similarity between "train" & "arrives"
- $0.1e_{arrives}$: Similarity between "arrives" & "arrives"
- $0.2e_{soon}$: Similarity between "soon" & "arrives"

Now, the similarity here is simply calculated using "Dot Product"

$$e^{new}_{arrives} = 0.3e_{the} + 0.4e_{train} + 0.1e_{arrives} + 0.2e_{soon}$$

The number here are the output from dotproduct.

$$0.3 = e_{the} \cdot e^{T}_{arrives}$$

$$0.4 = e_{train} \cdot e^{T}_{arrives}$$

} dot product

In simple terms we can represent it as:

$$e^{(new)}_{arrives} = [e_{the} \cdot e^{T}_{arrives}]e_{the} + [e_{train} \cdot e^{T}_{arrives}]e_{train} + [e_{arrives} \cdot e^{T}_{arrives}]e_{arrives} + [e_{soon} \cdot e^{T}_{arrives}]e_{soon}$$



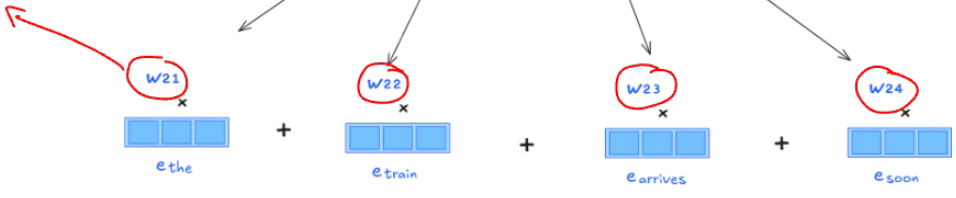
$$w_{21} = \frac{e^{s_{21}}}{e^{s_{21}} + e^{s_{22}} + e^{s_{23}} + e^{s_{24}}}$$

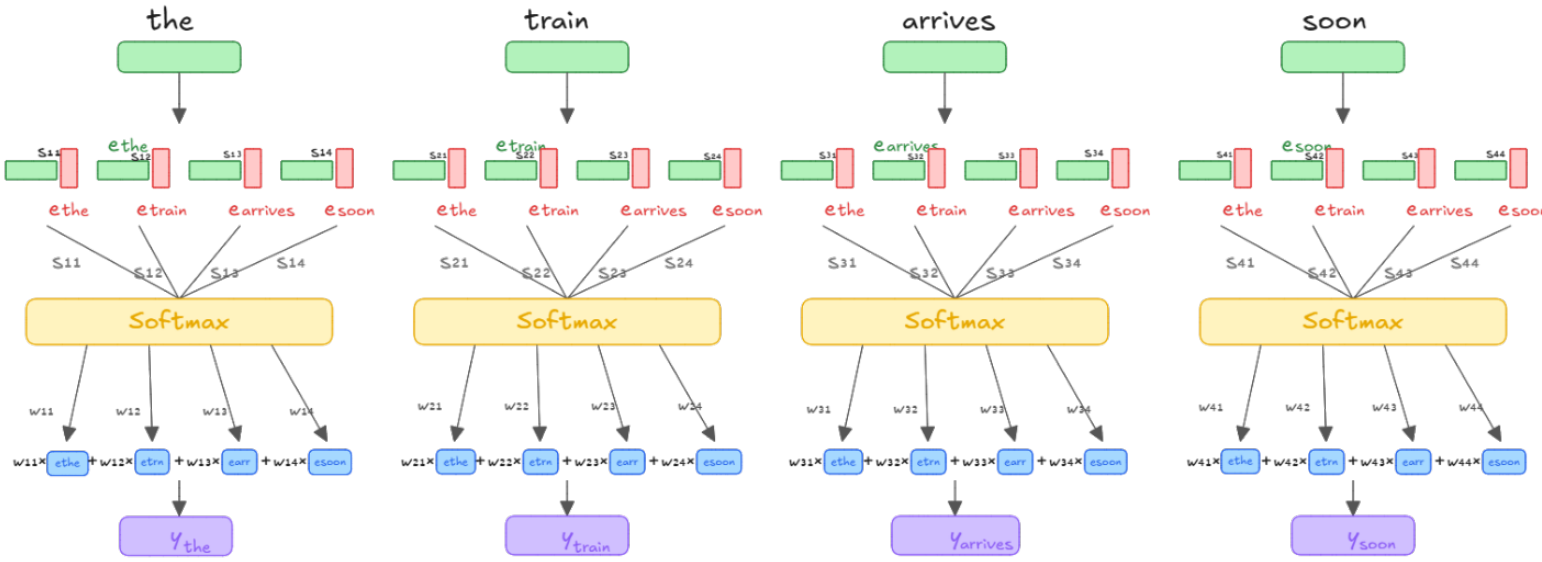
Normalization



Normalization layer

(Simply because DL models usually perform better (chances of -ve too))

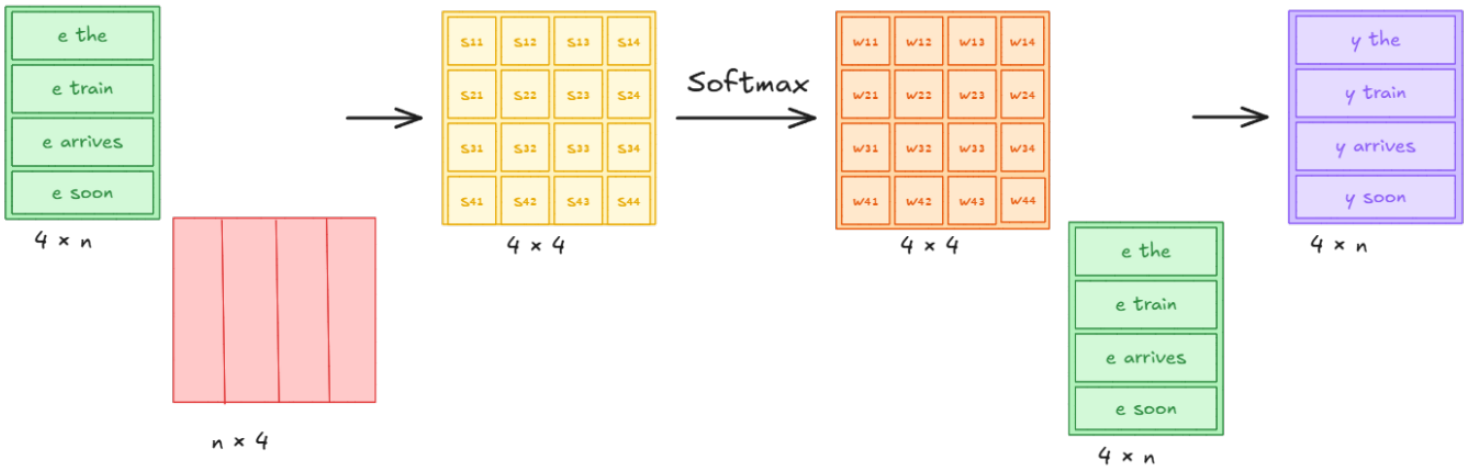




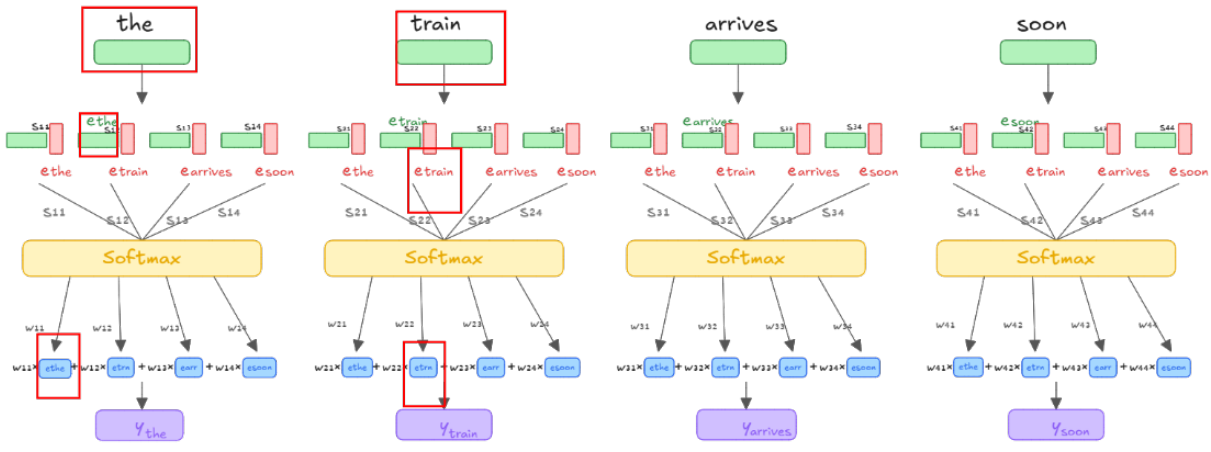
- These operations are parallel operations.
- There are no learning parameters involved.

parallel operations are simply done by matrix.

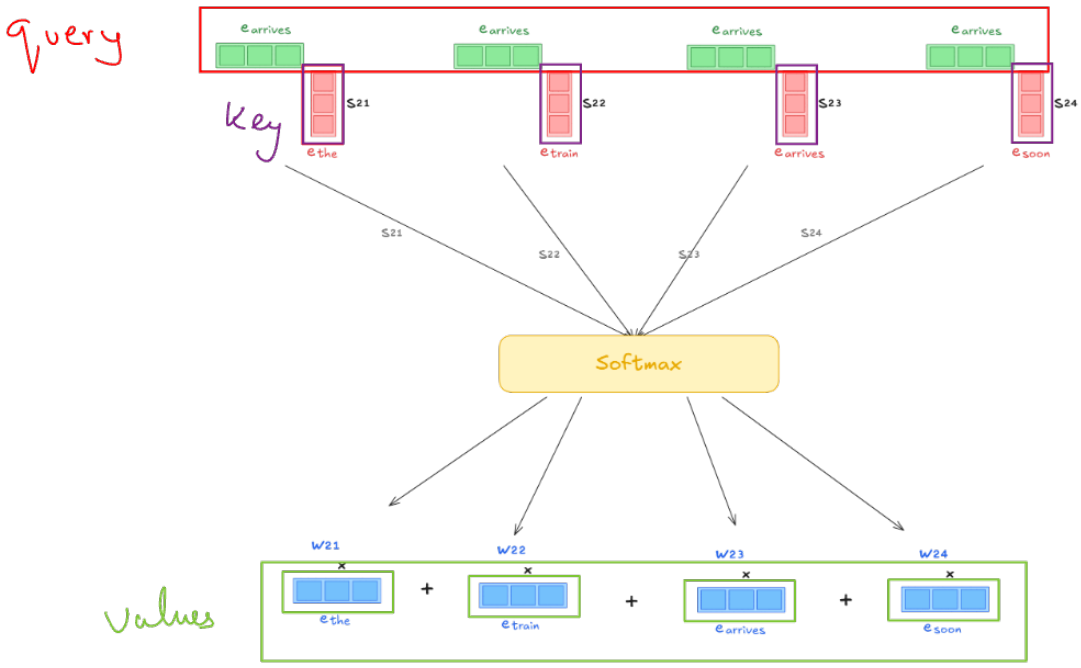
Since all the embeddings are vectors, we can use them to create a matrix.



If we take a close look, we can notice that each embeddings repeats 3 times for their specific words



This is where the concept of query key values comes in



Now, the reason we had to introduce this concept of key query value is because if we notice, for each word, one embedding repeats three time, → one is in initial phase ($e_{arrives}$) and another for dot product $e_{arrives}$. and third one after normalization $e_{arrives} \times W_{23}$.

We should note that everytime, these embeddings does not change! They are all same sent to each other. We do not want this.

Why? Imagine you are searching for a job, and you have to make a resume. Also imagine that you are author and you have wrote down your whole biography that has about 300 pages.

Now when a recruiter reaches out to you, he asks you for your resume. you have two options: either send him a resume or instead send him your whole biography to read that also contains your work experience and skills.

Now, it is obvious no one will read the whole book since they dont want to spend this much time.

Similarly, when an embeddings wants something that important to that embedding only, instead of sending the whole embedding (book) you can refine and send only the important part (resume).

Here, the job search is your Query

Resume is your Key

Resume's content is your Value

Query (Q) — *"What am I looking for?"*

The Query vector represents what a given token is *searching for* — what kind of contextual information it needs. For example, if the token is a verb, its query might encode something like "I need to find my subject." The query is produced by multiplying the token's embedding x_i by a learned weight matrix W_Q:

$$Q_i = x_i \cdot W_Q$$

Key (K) — *"What do I contain?"*

The Key vector represents what a token *offers* or *advertises* about itself. It's the label that other tokens compare their queries against. If a token is a noun, its key might encode "I am a subject." It's computed similarly:

$$K_i = x_i \cdot W_K$$

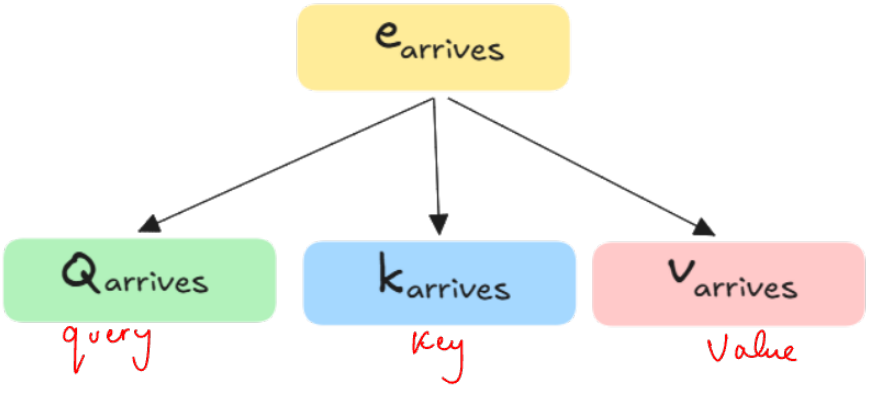
The dot product $Q_i \cdot K_j$ between a query and a key gives us the **compatibility score** — how relevant token j is to token i. High dot product means high relevance.

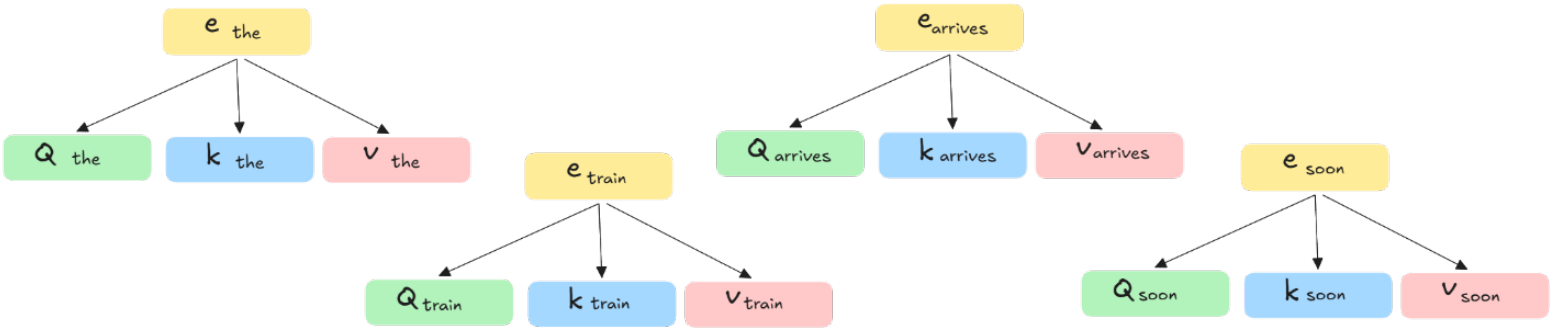
Value (V) — *"Here is my actual information."*

The Value vector is the *actual content* that gets passed forward once attention decides it's relevant. The key tells you *whether* to attend; the value tells you *what* to extract. It's computed as:

$$V_i = x_i \cdot W_V$$

Now, key, query and Value are seperate vectors derived from the parent embedding





for all embeddings

Now the question is that how are these Q, K, V vectors calculated from one vector?

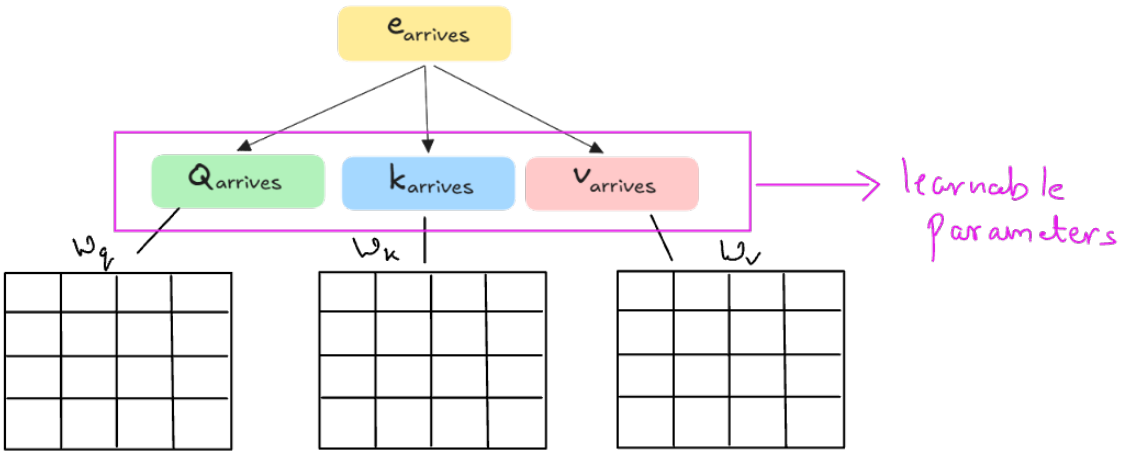
Since we know that all of these vectors are originated from the embeddings, we need to take the help of the data

Back propagation

If we take our resume example, we can first experiment. If the recruiter does not reach out, we change our resume, if they dont like something we change that and overtime, we have a perfect resume

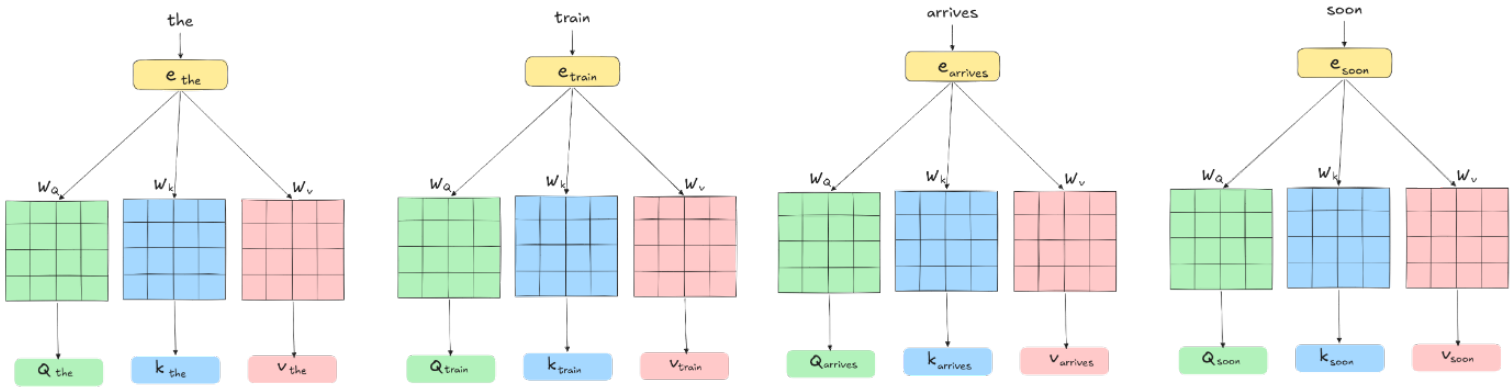
We use linear transformation to get values of Q, K, V.

for ex,

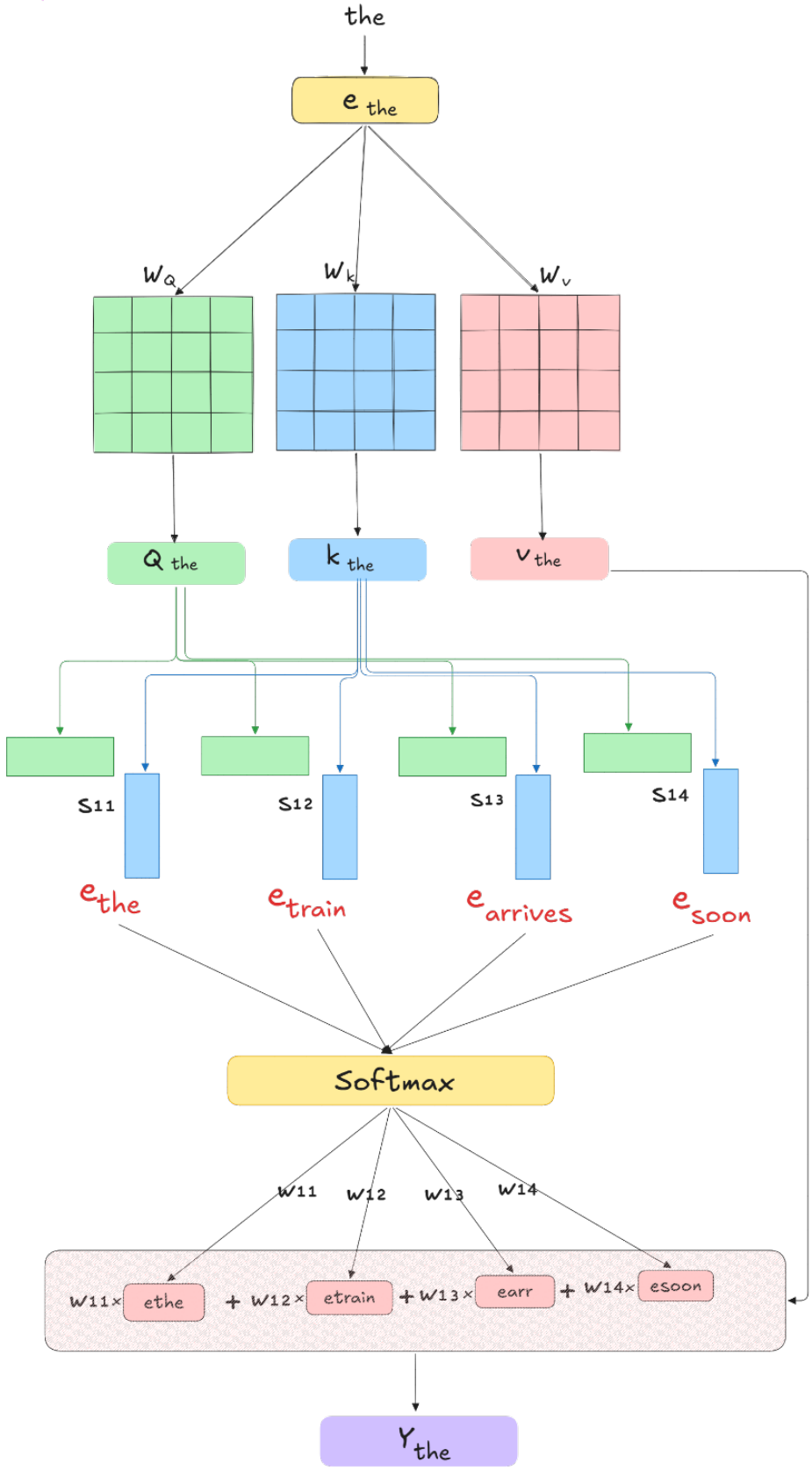


How are the values inside these W_q, W_k and W_v matrix decided?

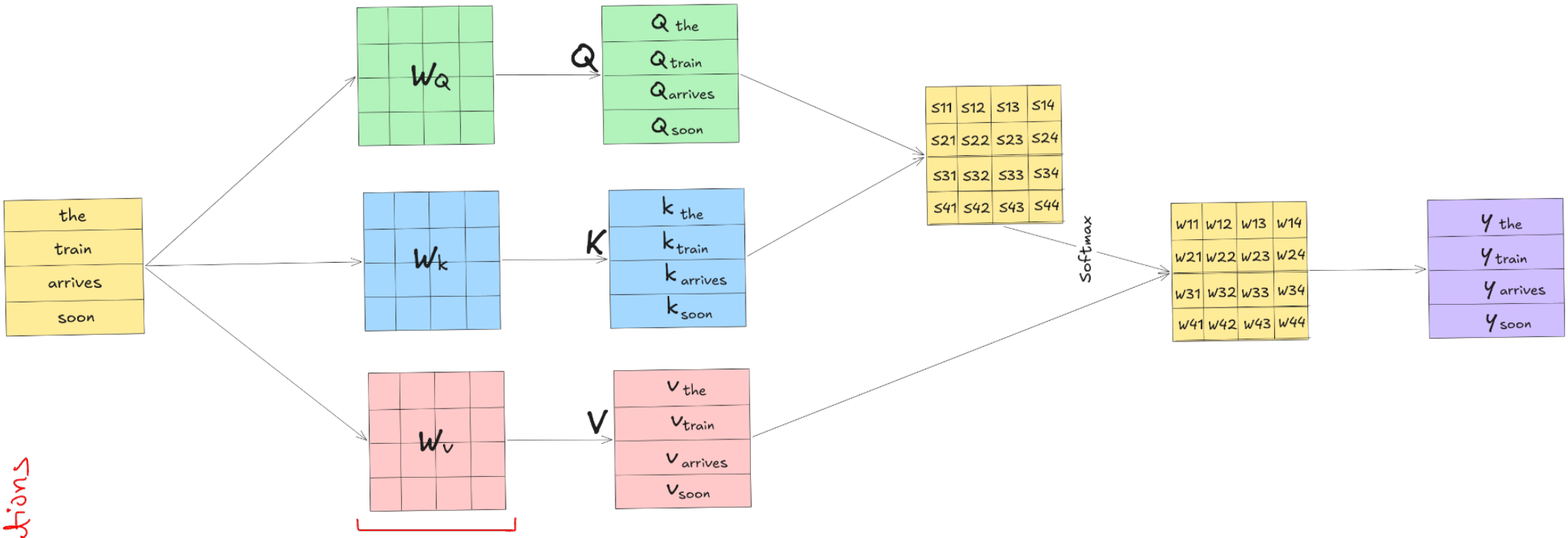
This is where we have learnable parameters. We start with random values and let the optimizer use backprop to update the values!



finally, the whole diagram



(Same for other words in the seq)



Same for all sequences (Matrix)

learnable parameters

Parallel operations

We can express this in mathematical form:

$$\text{Attention}(Q, K, V) = \text{Softmax}(QK^T)V$$

But in the research paper, there's one more term used

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

(Scaled dot product)

d_k is the dimension of the key vector

The simple reason behind this is that lower dimension vector has less variance whereas, higher dim has higher variance

And because the softmax simply gives us the probability, it tends to focus more on the larger values and ignores the small values or overtime small values becomes smaller and smaller which may lead to vanishing gradients.

Softmax uses exponents which is why if a very large value is given, it will assign it a very high prob (99%) and other lower (1%) (for ex.)

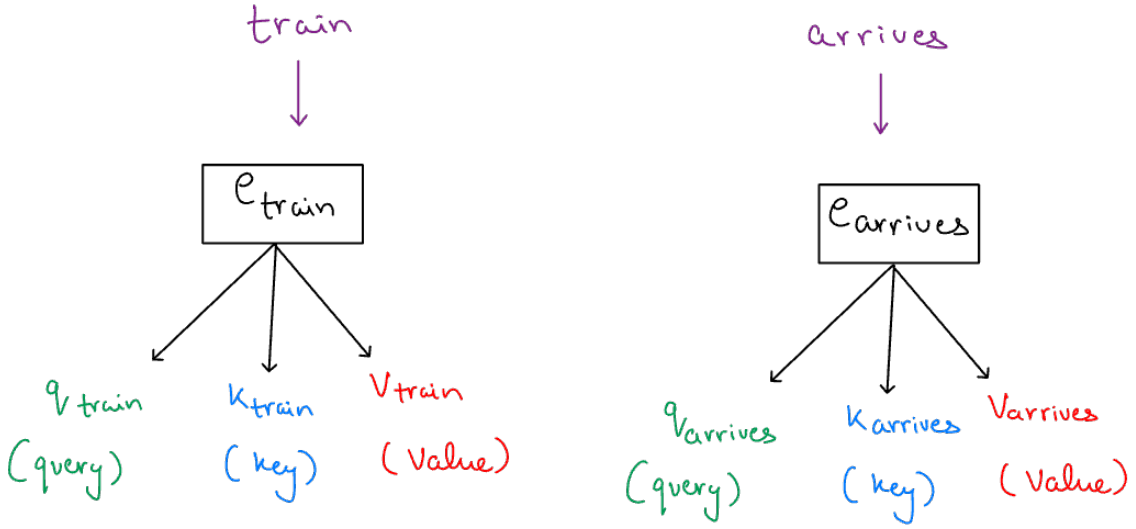
So researchers come up with $\sqrt{d_k}$.

The math is a bit long, but we can just take $\sqrt{d_k}$ and divide it in the formula.

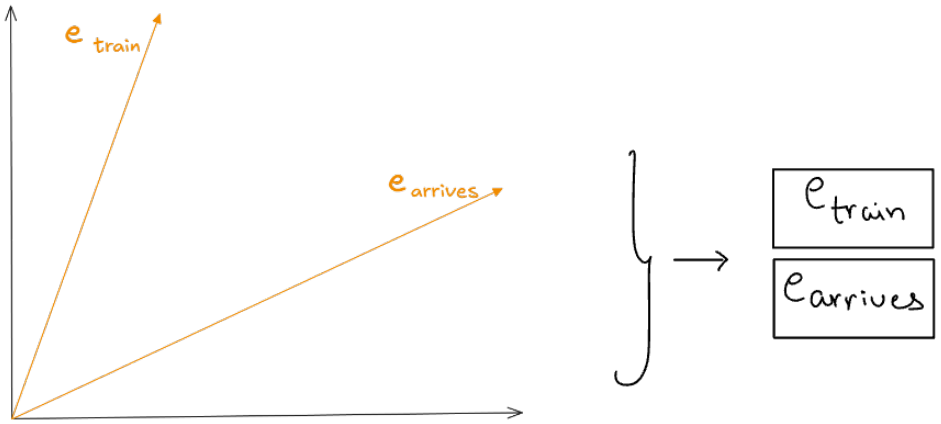
$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

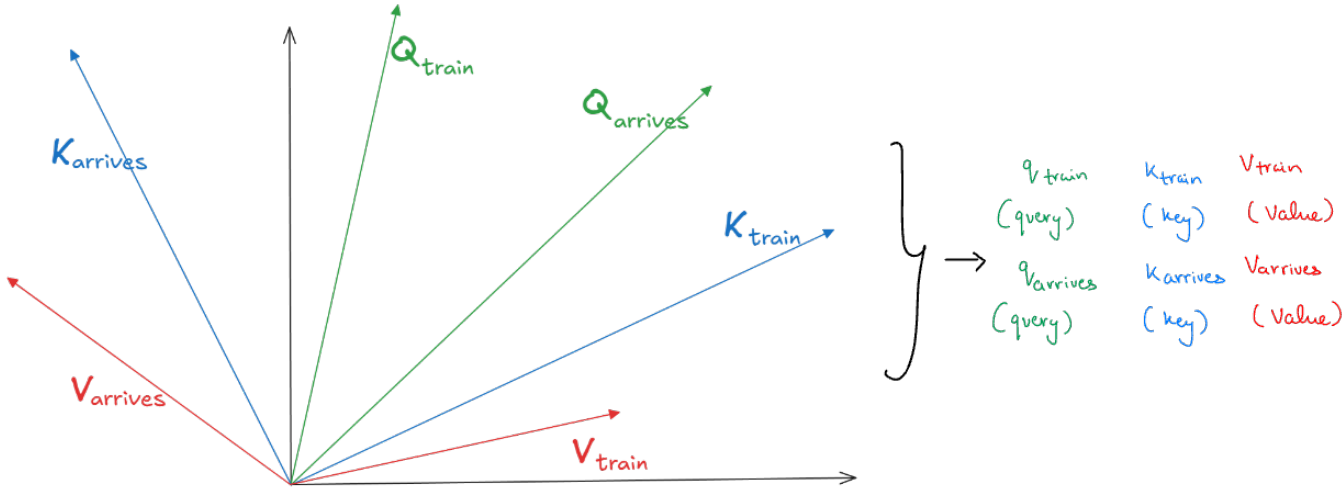
Geometric Intuition

If we take two words from a sequence : "train", "arrives"
we firstly convert the words into static embeddings
then we create three vectors for both separate words
query vector, key vector, Value Vector



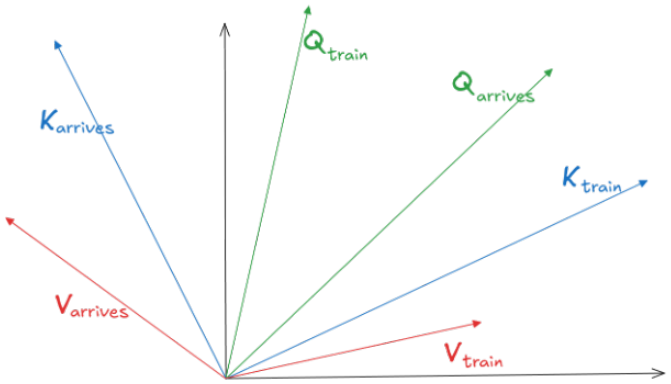
Now, lets plot them first step by step





Now once we have the Q, K, V vectors, we scale them by $\sqrt{d_k}$.

Assume that the graph below now has scaled down the vectors

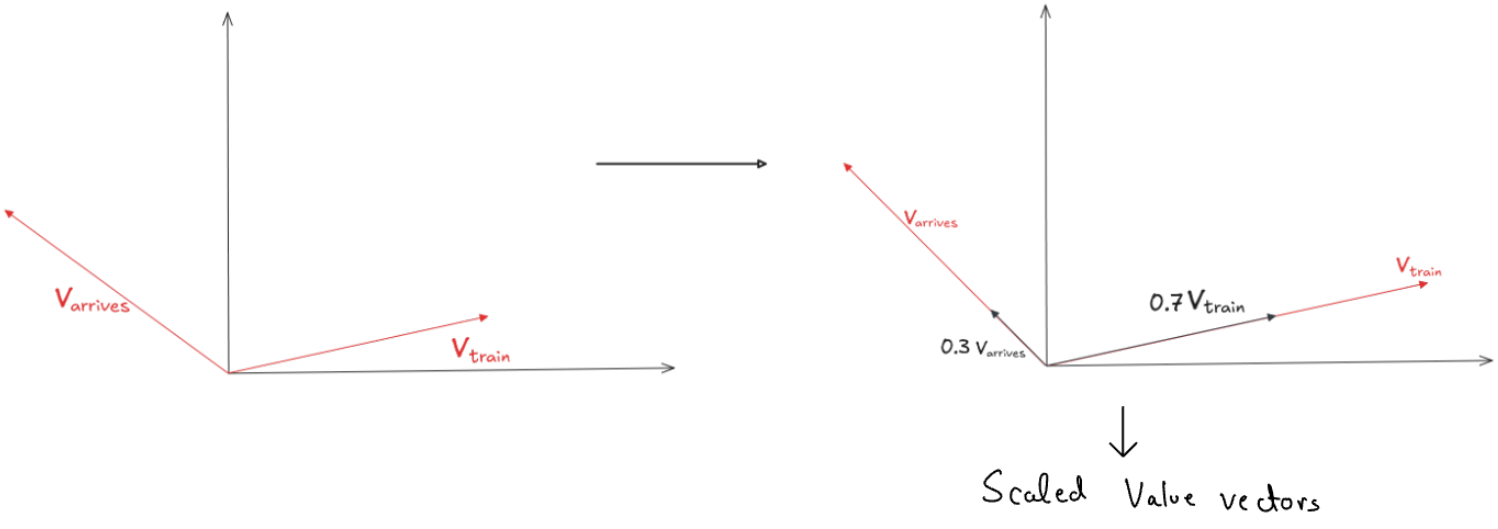


Now we apply softmax on the vectors to get probability.

Once we have the probability, we can use that as w_{ij} (scalar, probability from softmax) $\times$ V vector to scale those value vectors

for ex : $w_{11} = 0.3$ (train), $w_{12} = 0.7$ (arrives)

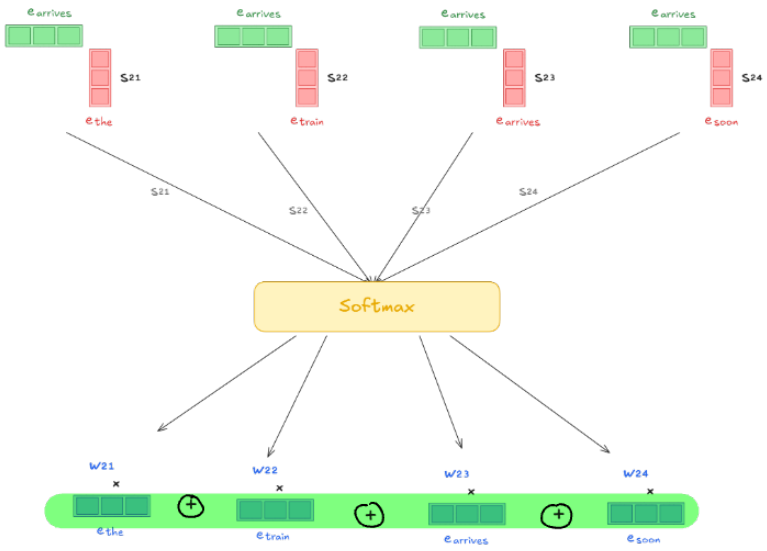
So, we scale value vectors of train and arrives by respective values (0.3, 0.7)

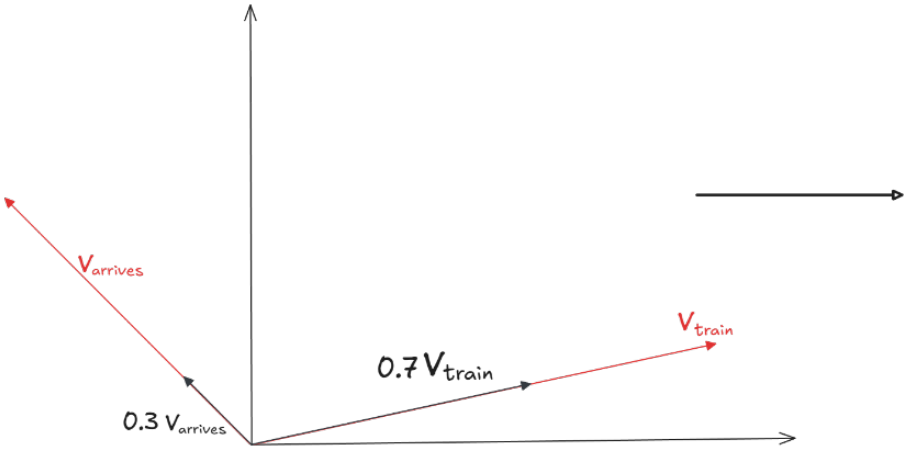


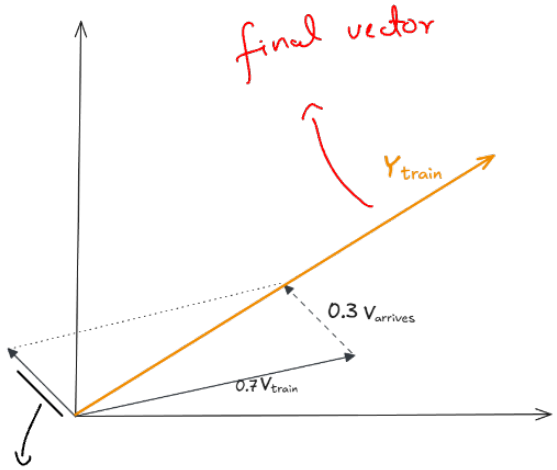
↓

Scaled Value vectors

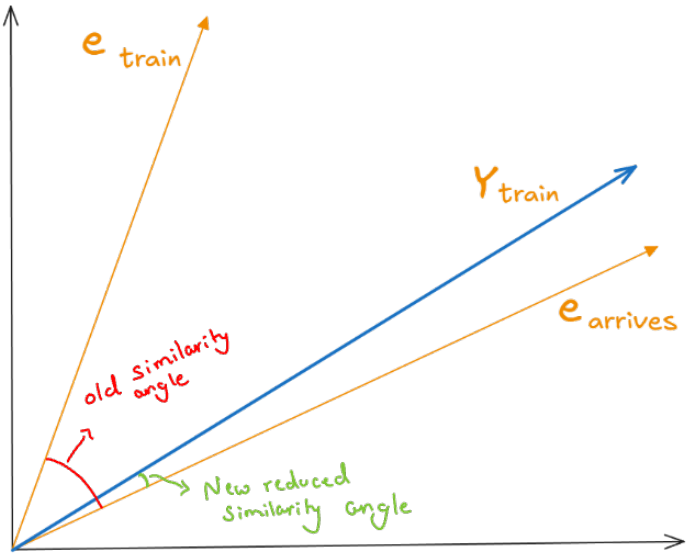
Once we have the value vector scaled by softmax, we now have to add the final vectors







Adding this vector ($V_{arrives}$) to V_{train} by vector addition.

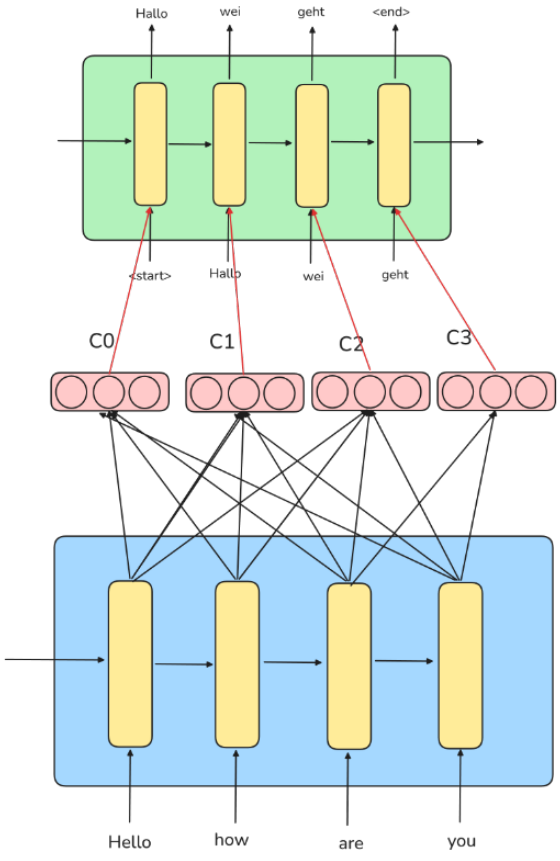


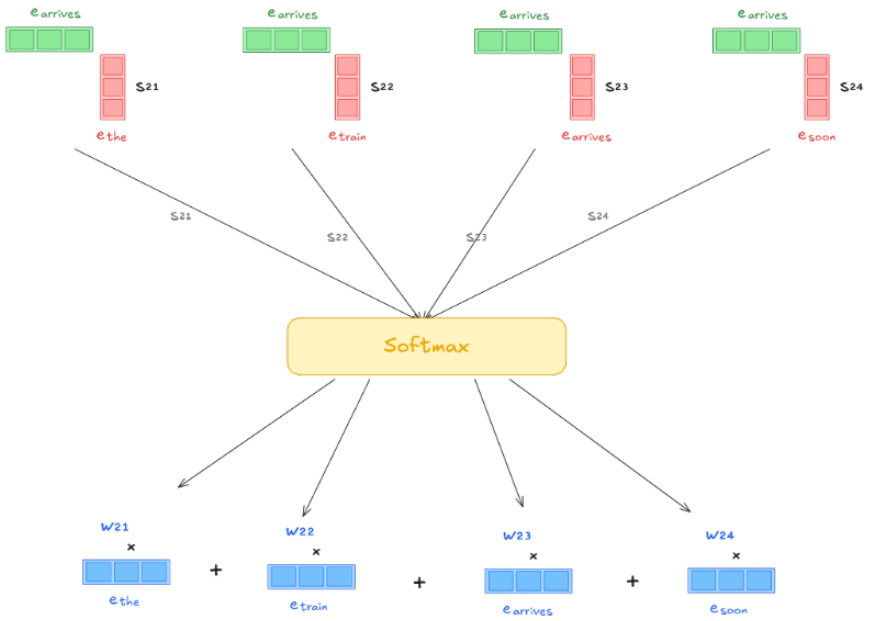
finally, we can notice that Y_{train} and $e_{arrives}$ end up close to each other. This is how self attention works geometrically for similar "context".

Why named 'Self attention'?

In classic encoder-decoder attention, the decoder hidden state is the query, the encoder's hidden state are the key/values and you compute alignment score between each query and each key, softmax them, and take a weighted sum of values as a context vector.

Self attention keeps this exact same pattern - query-key-scoring, Softmax normalization, weighted sum of values. So conceptually it is still an attention mechanism, just applied in different configuration.





In cross-attention, queries come from one sequence (decoder: german), while keys/values comes from another (encoder: english), so one sequence attends to another.

In self attention, all three Q, V, K, are computed from the same sequence (english), so every token representation is updated by looking at all tokens in that same sequence (including itself)

The problem with Self Attention

In self attention, for every token in a sequence, we compute one set of Query, key and Value vectors. The query of one token is matched against the keys of all other tokens to produce attention weights, and those weights are used to create a weighted combination of values. This gives each token a "Context Aware" representation.

The core limitation with this approach is that it only has 'One perspective'. Therefore, a single attention computation can only learn one pattern of relationship at a time.

for example 1. John promised bill to finish the work before he left. (Who is "he"?)

2. The trophy doesn't fit into the suitcase because it's too large (Whats large? suitcase or trophy?)

What does "it" refer to gramatically? Whats the action relationship? Whats the casual link? A single set of K, Q, V matrices Compresses all of this into one weighted average. It has to make a compromise that leads to lose information.

The above examples are too ambiguous and can have multiple different meaning depending on the context / Perspective.

This is where we introduce the concept of Multi Head Attention which simply has multiple perspectives.

Multi Head Attention Mechanism

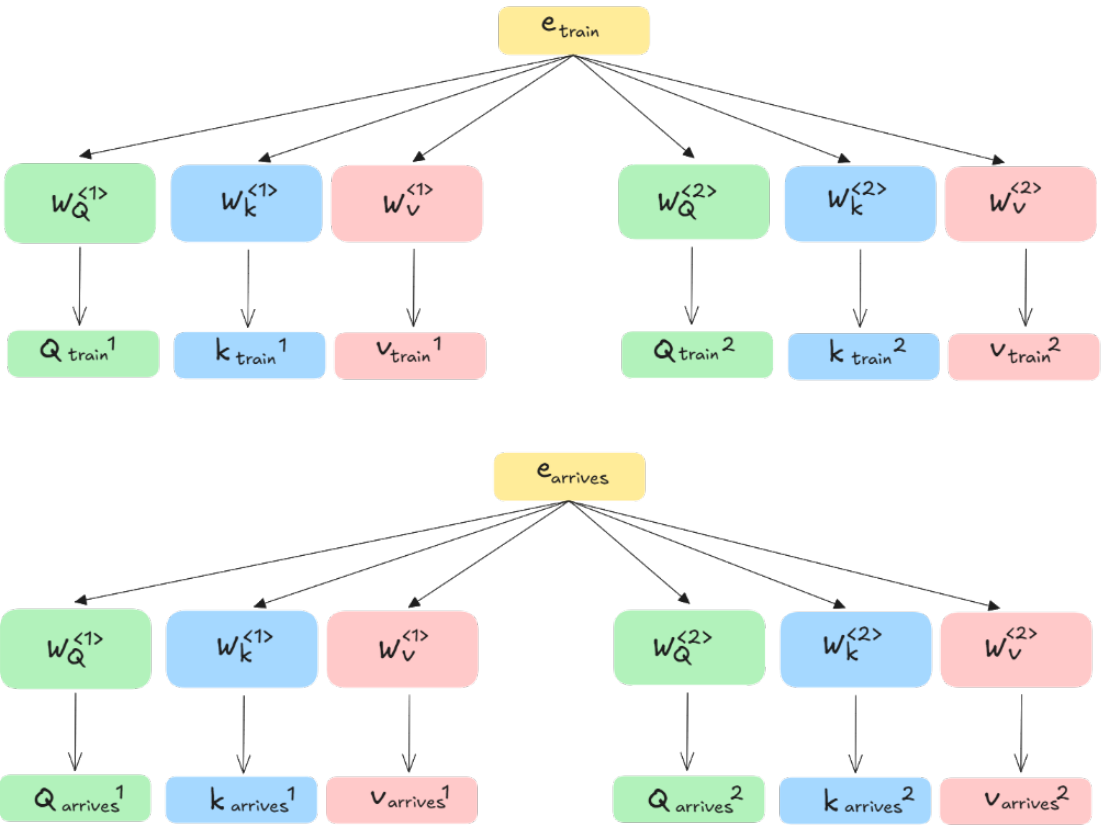
Multi head Attention is a mechanism where instead of performing a single attention function over the full d_{model} dimensional space, we linearly project the queries, keys and values h times using different learned projection matrices.

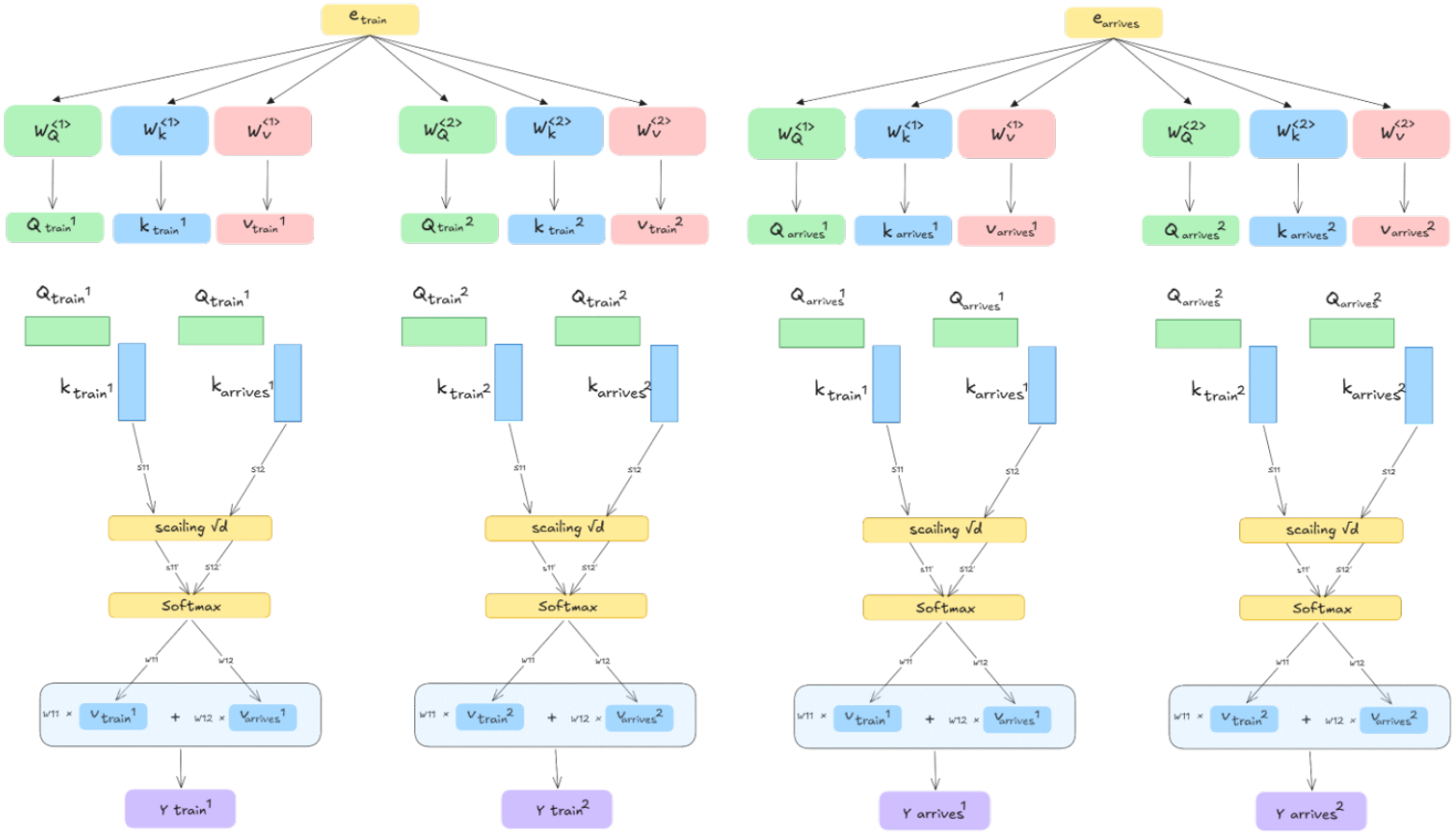
The core idea is that each head gets its own set of W_i^Q, W_i^k, W_i^V weights, so each head learns to attend to different types of relationships (syntatic, semantic, positional etc) independently. The concatenation and final W^O projection then combine all these perspectives into a single rich representation.

In short : single head attention asks one question about each token pair.

Multihead attention asks h different questions in parallel, then merges the answers.

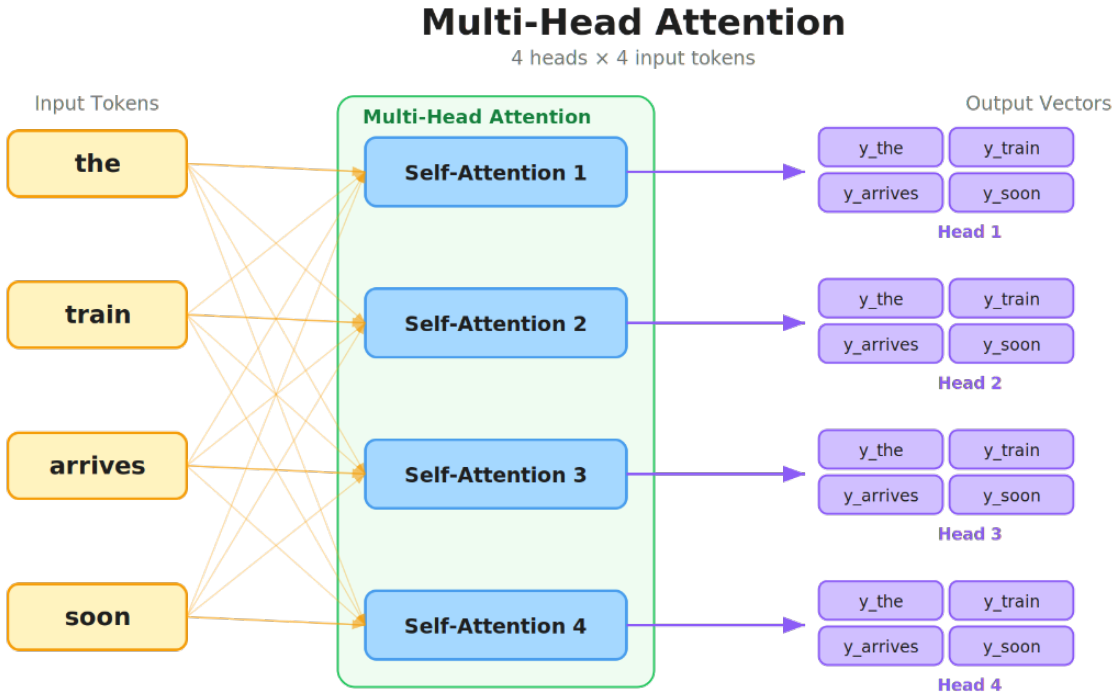
for ex, "train", "Arrives"





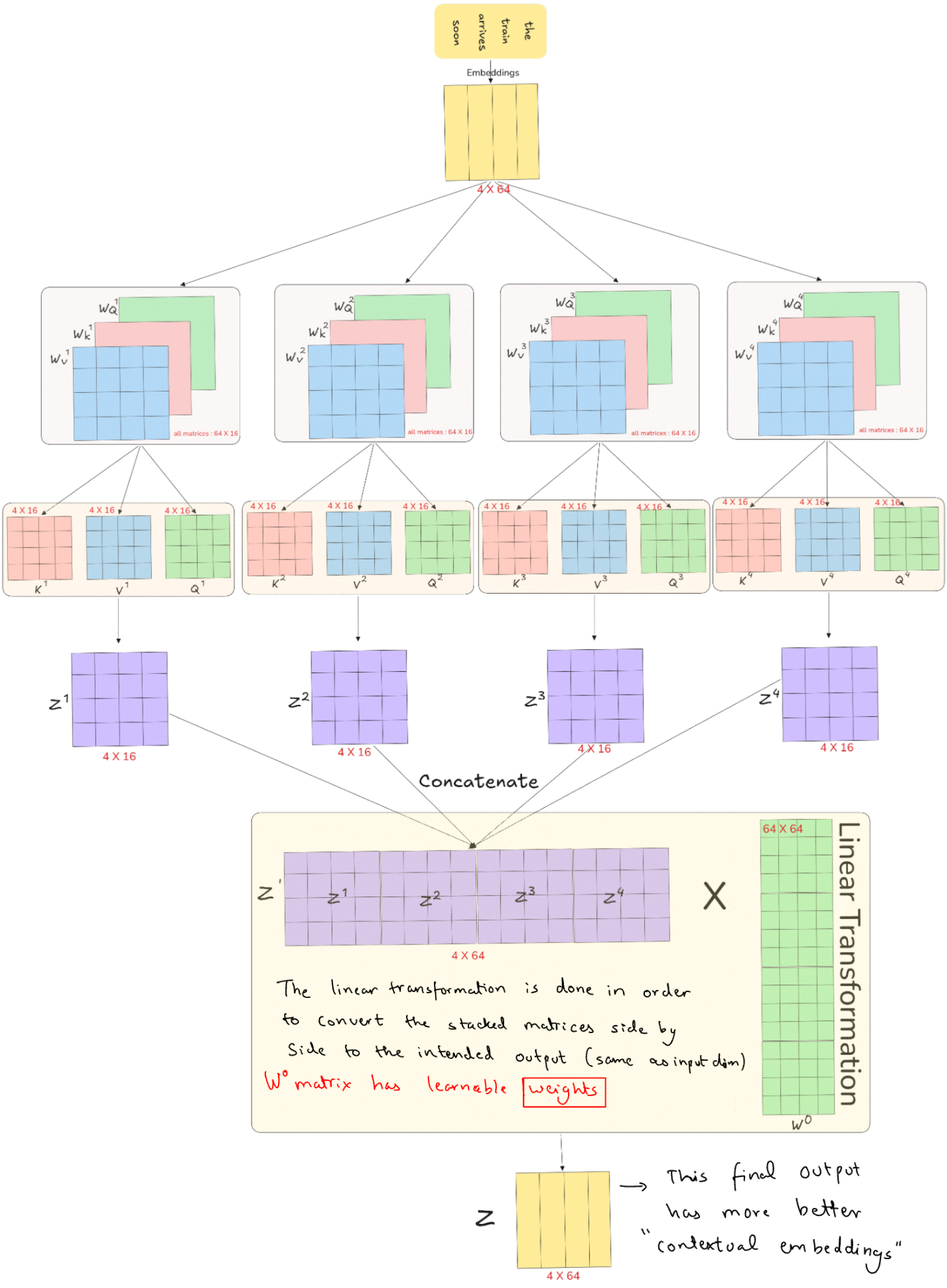
Here, as we can see, we used two attention heads.

The y_{train}^1, y_{train}^2 and $y_{arrives}^1$, $y_{arrives}^2$ are later concatenated in order to get the output same as input dim.



Here, we are capturing 4 different perspectives

The matrix Calculation



The problem with Self Attention

Self attention computes relationship between all pairs of tokens simultaneously. It treats the input as a set, not a sequence. If we shuffle the words in a sentence, the attention scores between any two specific words remain exactly the same. The mechanism has no way of knowing which word came 1st, 2nd or last.

for ex. "The cat sat on the mat"
"mat the on sat cat the"

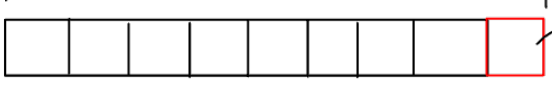
Self attention would produce the same attention weights between "cat" and "sat" in both cases.
But obviously the word order matters for meaning
This is the core motivation for Positional Encoding.
We need to inject the positions into the model.

Instead of directly jumping to positional encoding topic, lets approach it from the base.

We know the we somehow need the model to know what word came in what position of the sequence.
The basic way to solve this is to add a counter to all the words.

"My Name is Vraj"
1 2 3 4

We can simply add these counter numbers in the respective embeddings. Just append it at the last position.



Problem : The problem with the previous counter approach is that it is 'Unbounded'. Meaning those counter numbers have no upper limit.

If a sequence has 10,000 words, while training the model, it will experience "Exploding Gradients".

In order to fix the exploding gradients, we can scale those counter numbers in range 0-1. By simply divinding all the counter numbers with the largest counter number.

$\frac{1}{6}, \frac{2}{6}, \frac{3}{6}, \frac{4}{6}, \frac{5}{6}, \frac{6}{6}$ } 0-1 range

Now the problem is that these numbers are Discrete and neural networks perform good with continuous numbers.

The biggest issue is also that the model still does not know the distance (gap) between two words. This is called as relative position.

The model does not know that "Vray" is at 4th position while "name" is at 2nd.

Note that we cannot do 4-2 = 2 here, because when we embedd these counter values in an embedding, it goes through non-linear layers through the network which destroys the linear (4-2) relationship. The model cannot recover it.

We have 3 big problems till now:

① Unbounded ② Discrete ③ Relative position.

Based on the problems, we need something that has:

1. Bounded
2. Continuous
3. Periodic Position (explained below)

Based on all the requirements, trignometric functions are the best fit.

Periodic Position: What we really want is an encoding technique where the relationship between any two positions seperated by a fixed distance k always looks the same, regardless of where in the sequence those positions are.

That is, the pattern that says "these 2 tokens are 3 apart", should be the same whether it's position (1, 4) or (5, 50).

This is what sinusodial encodings achieve.

A periodic function is one that repeats its values at regular intervals. Sine and Cosine are periodic

So, lets use Sine encodings.

Instead of those counter numbers, we use Sin (pos) equation

$pos\,1 = Sin(1) = 0.841$

$pos\,2 = Sin(2) = 0.909$

$pos\,3 = Sin(3) = 0.141$

This is almost perfect but since Sine is periodic $2\pi = 6.28$ So, $Sin(0) \approx Sin(6) \approx Sin(13)$ .... So these positions will be nearly identical or sometimes the same.

To avoid this problem, we add cosine function as well. So now instead of scalar positional encoded value, we will use a vector.

pos 0 → [Sin(0), Cos(0)] = [0.00, 1.00]
pos 1 → [Sin(1), Cos(1)] = [0.841, 0.540]
pos 2 → [Sin(2), Cos(2)] = [0.343, 0.891]

This helps but the fundamental issue of repeated encoding still remains. There is still some chances that few encodings can be same. We can add more Sin and Cosine functions, but there is still some probability of an encoding being repeated.

Therefore we use "Multiple Frequencies"

Instead of single Sin / cosine pair, use many pairs at different frequencies. Some oscillate quickly (distinguishing nearby positions), others oscillate slowly (distinguishing distant positions).
Together, they create a unique "fingerprint" for each position.

The encoding vector at each position has the same dimensionality as the embedding (d-model), filled with alternating sin and cos values at geometrically spaced frequencies.

The formula Researchers came up with in the "original paper"

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_model}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_model}}\right)$$

pos : position of token in sequence

i : the dimension index

d_model: dimensionality of embeddings (ex. 512)

Even indexed dimensions (0, 2, 4, ...) use Sine

odd indexed dimensions (1, 3, 5, ...) use Cosine

The term $10000^{2i/d_model}$ controls the wavelength

This wavelength forms a geometric progression from 2π to $2\pi \times 10{,}000$. This spread of frequencies makes sure that no two positions within any practical sequence length share the same encoding vector.

"The train arrives soon" $\{ d_model = 4$

"The"

Pos = 0

i = 0 $PE(0,0) = \sin(0/10000^{0/4}) = 0.00$

i = 1 $PE(0,1) = \cos(0/10000^{0/4}) = 1.00$

i = 2 $PE(0,2) = \sin(0/10000^{2/4}) = 0.00$

i = 3 $PE(0,3) = \cos(0/10000^{2/4}) = 1.00$

$$PE(0) = [0.00, 1.00, 0.00, 1.00]$$

"train"

(pos = 1)

i = 0 $PE(1,0) = \sin(1 / 10000^{1/1}) = 0.8415$

i = 1 $PE(1,1) = \cos(1 / 10000^{1/1}) = 0.5403$

i = 2 $PE(1,2) = \sin(1 / 10000^{1/100}) = 0.0100$

i = 3 $PE(1,3) = \cos(1 / 10000^{1/100}) = 0.999$

$PE(0) = [0.8415, 0.5403, 0.0100, 0.999]$

"arrives"

(pos = 2)

i = 0 $PE(2,0) = \sin(2 / 10000^{2/1}) = 0.9093$

i = 1 $PE(2,1) = \cos(2 / 10000^{2/1}) = -0.4161$

i = 2 $PE(2,2) = \sin(2 / 10000^{2/100}) = 0.0200$

i = 3 $PE(2,3) = \cos(2 / 10000^{2/100}) = 0.9998$

$PE(0) = [0.9093, -0.4161, 0.0200, 0.9998]$

"soon"

(pos = 3)

i = 0 $PE(3,0) = \sin(3 / 10000^{3/1}) = 0.1411$

i = 1 $PE(3,1) = \cos(3 / 10000^{3/1}) = 0.9900$

i = 2 $PE(3,2) = \sin(3 / 10000^{3/100}) = 0.300$

i = 3 $PE(3,3) = \cos(3 / 10000^{3/100}) = 0.9996$

$PE(0) = [0.1411, -0.9900, 0.300, 0.996]$

Positional Encoded Matrix

0.000	1.000	0.000	1.000
0.8415	0.5403	0.0100	0.999
0.9093	-0.4161	0.0200	0.998
0.1411	-0.9900	0.300	0.996

Now once we have the positional encoded vectors for each words, we simply add them with their particular embeddings

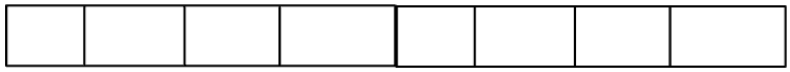
Note that we do not concatenate these embeddings because doing so will increase the parameters and slow down the training process



Original Embedding Positional encoded vector



Concatenate ✗ (This is wrong method



This will double the total number of parameters

Instead, we add both vectors.

Word embedding + Positional Embedded Vector

for ex. train =

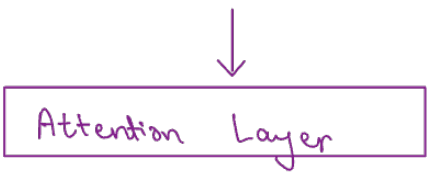
⊕ PE vector =

0.89	0.23	-0.56	0.44
0.8451	0.5403	0.0100	0.9999

Vector addition

✓ Correct way

1.731	0.773	-0.550	1.439



Attention Layer

The final vector addition of embedding + PE vector results in final embedding that now contains both positional and semantic info. Model learns to seperate both positional and semantic information from the summed vector.

The Linear Relationship Property

(Relative Position Encoding via Delta)
Based on: *"Linear Relationships in the Transformer's Positional Encoding"* — Timo Denk's blog

The Core Claim

For any fixed offset **k**, there exists a **linear transformation** (a matrix M_k) such that:

$$\mathbf{PE(pos + k) = M_k \cdot PE(pos)}$$

Crucially, M_k depends **only on k** (the distance), **not on pos** (the absolute position). The same matrix transforms position 5 → 8 (k=3) as position 100 → 103 (k=3).

How It Works (single sin/cos pair at frequency ω)

Consider one pair of dimensions with frequency ω:

```
PE(pos)      =  [ sin(ωp),      cos(ωp)      ]
PE(pos + k)  =  [ sin(ω(p+k)),  cos(ω(p+k))  ]
```

Applying the trigonometric addition formulas:

```
sin(ω(p+k))  =  sin(ωp)·cos(ωk) + cos(ωp)·sin(ωk)
cos(ω(p+k))  =  cos(ωp)·cos(ωk) − sin(ωp)·sin(ωk)
```

This can be written as a matrix multiplication:

```
┌              ┐     ┌                     ┐   ┌        ┐
│ sin(ω(p+k))  │     │  cos(ωk)   sin(ωk)  │   │ sin(ωp)│
│              │  =  │                     │ · │        │
│ cos(ω(p+k))  │     │ −sin(ωk)   cos(ωk)  │   │ cos(ωp)│
└              ┘     └                     ┘   └        ┘
```

That 2×2 matrix is a **rotation matrix** — it depends only on **k** and **ω**, not on pos.

Why This Matters

Since the transformation from PE(pos) to PE(pos + k) is **linear**, the model's attention mechanism (which is itself based on linear projections followed by dot products) can easily learn to detect relative positions. A single learned linear layer can convert "here are two position encodings" into "these are k positions apart."

The model doesn't need to learn complex non-linear functions to recover distance information — it's right there in a form that linear algebra can extract directly.

> **Why integer counters fail:** With integers, after non-linear transformations in the network, the subtraction relationship ($pos_2 - pos_1$ = distance) is destroyed. With sinusoidal encodings, the relative position information is preserved as a linear (rotation) relationship that survives and can be directly leveraged by the linear projections in attention.

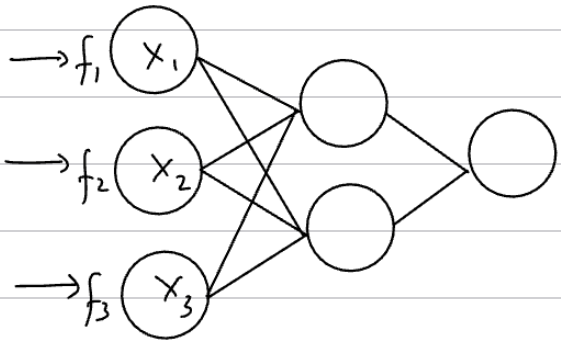
Layer Normalization

Normalization in deep learning refers to the process of transforming data or model outputs to have specific statistical properties, typically a mean of zero and a variance of one.

What do we normalise?

① The inputs (f_1, f_2, f_3)

② The activations of hidden layers



Benefits of applying Normalization in DL

Improved training Stability : It helps to stabilize and accelerate the training process by likelihood of extreme values that can cause gradients to explode or vanish.

Faster Convergence : By normalizing inputs or activations, models can converge more quickly because the gradients have more consistent magnitudes. This allows for more stable updates during back prop.

Regularization Effects : Some normalization techniques like batch normalization introduces a slight regularizing effect by adding noise to the mini-batches training. This can help reduce overfitting.

Reduces Internal Covariate Shift

Covariate Shift: It occurs when the distribution of input features (x) changes between training and test data, but the relationship between inputs and outputs stays the same

Example: You train a model to recognize cats using indoor photos but test it on outdoor photos. The lighting, backgrounds and colors are different (input distribution changed) but a cat is still a cat (underlying relationship is the same.)

This makes the model perform poorly because it has not seen data from the new distribution during training.

Internal Covariate Shift

It is the same concept but happening inside the network between layers during trainings.

As the network trains and weights update, the distribution of inputs to each layer keeps changing. For example:

- Layer 2 recieves input from Layer 1
- As Layer 1's weights update, the distribution of its outputs changes
- This means Layer 2 is constantly trying to learn from a "moving target"

Why is this a problem?

- Each layer must continuously adapt to new input distributions
- Slows down training
- Forces you to use smaller learning rate
- Makes the network harder to train.

How batch normalization helps: By normalizing inputs at each layer, it reduces how much distributions shift around (makes sure that the data stays normalized: $\mu = 0$, $\sigma = 1$), giving each layer a more stable foundation to learn from. This makes the layer focus on learning the actual patterns rather than constantly adapting to distribution changes.

Revisiting BN and how it works while training.

We use mini-Batch Gradient descent.

① firstly, a batch of data points is forward propagated, and it's mean and variance is computed for that Batch data points.

$$\mu = \frac{1}{m}\sum_{i=1}^{m} z_{11}^{i} \quad , \quad \sigma = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(z_{11}^{i} - \mu_B)}$$

This calculation of μ and σ is always done for each neuron seperately.

② Once that's done, we normalize

$$z^{i} = \frac{z^{i} - \mu_B}{\sigma_B + (\epsilon)}$$

→ The epsilon is a small constant added for numerical stability to prevent zero.

③ Scale and Shift

$$y_i = \gamma \hat{x}_i + \beta$$

Here γ (gamma) and β (Beta) are learnable parameters (internally managed by libraries) that allow the network to undo the normalization if needed. This is crucial because sometimes the optimal representation is not always zero mean and unit-variance (normalized: $\mu = 0$, $\sigma = 1$).
Sometimes, the model does not want us to normalize each layer's output which is why, if the model want to have another distribution, it can scale it using these learnable parameters)

During Test :

Since while training, we were using batches and normalizing the neuron's output using the batch's mean and variance, but while testing, there is only one data point at a time, so we cant calculate μ and σ?

This is where the concept of Exponentially weighted moving average comes. (EWMA)

During training, we forward propagate batches. The model's internal system maintains the EWMA using the means of each batches $\mu_{11}, \mu_{22} \ldots \mu_{nn}$.

Same goes for σ (variance). each batch has its different σ and μ, and by this, EWMA is maintained.

Now, when its time for training, we send out one data point at a time, the last value for the EWMA mean and EWMA variance is used this time as μ and σ for that data point

Therefore, while training a model, the model maintains a total of 4 parameters values. $\gamma, \beta, \mu, \sigma$

γ, β: learnable parameters

μ, σ: non learnable parameters

Advantages of BN

1. Training is more stable which enables us to select a better range of hyperparameters
2. faster training time ⟶ higher learning rate
3. Acts as a Regularizer ⟶ (not that strong, but a side +ve impact)
4. Reduces the impact of Batch Normalization.
5. Faster Convergence

Disadvantages

1. Batch size dependency : Performance degrades with small batches
2. Not suitable for RNN's
3. Adds Complexity : Extra hyperparameters and computational cost per layer
4. Memory Overhead : Requires additional parameters to maintain such as $\gamma, \beta, \mu, \sigma$, EWMA
5. Training/inference discrepancy : Uses batch statistics during training but running averages during inference, causing potential behaviour changes.

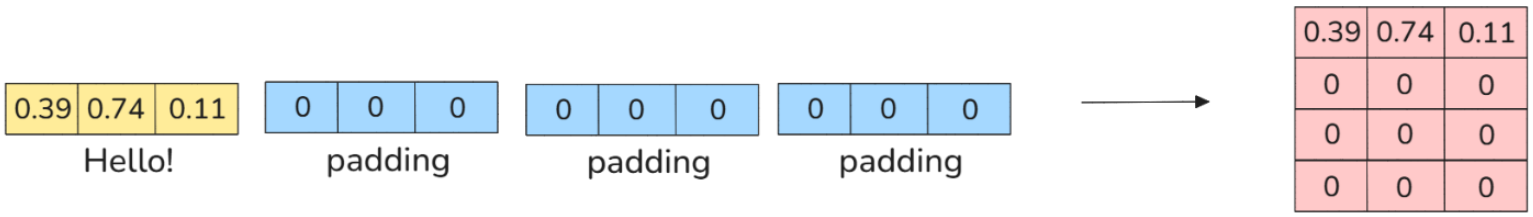
Why we dont use Batch Norm in Transformers?

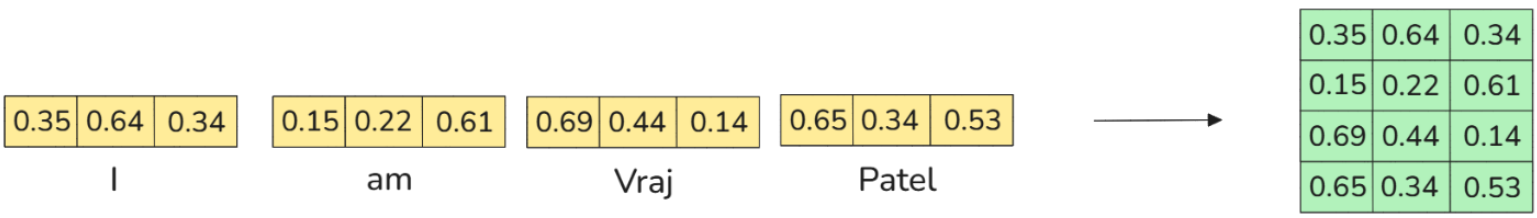
lets start with an example :

Review	Sentiment
Hello!	0
I am Vraj Patel	1
I love Tennis	1
I dont like pickleball	0

Embedding Dimension = 3
Batch Size = 2





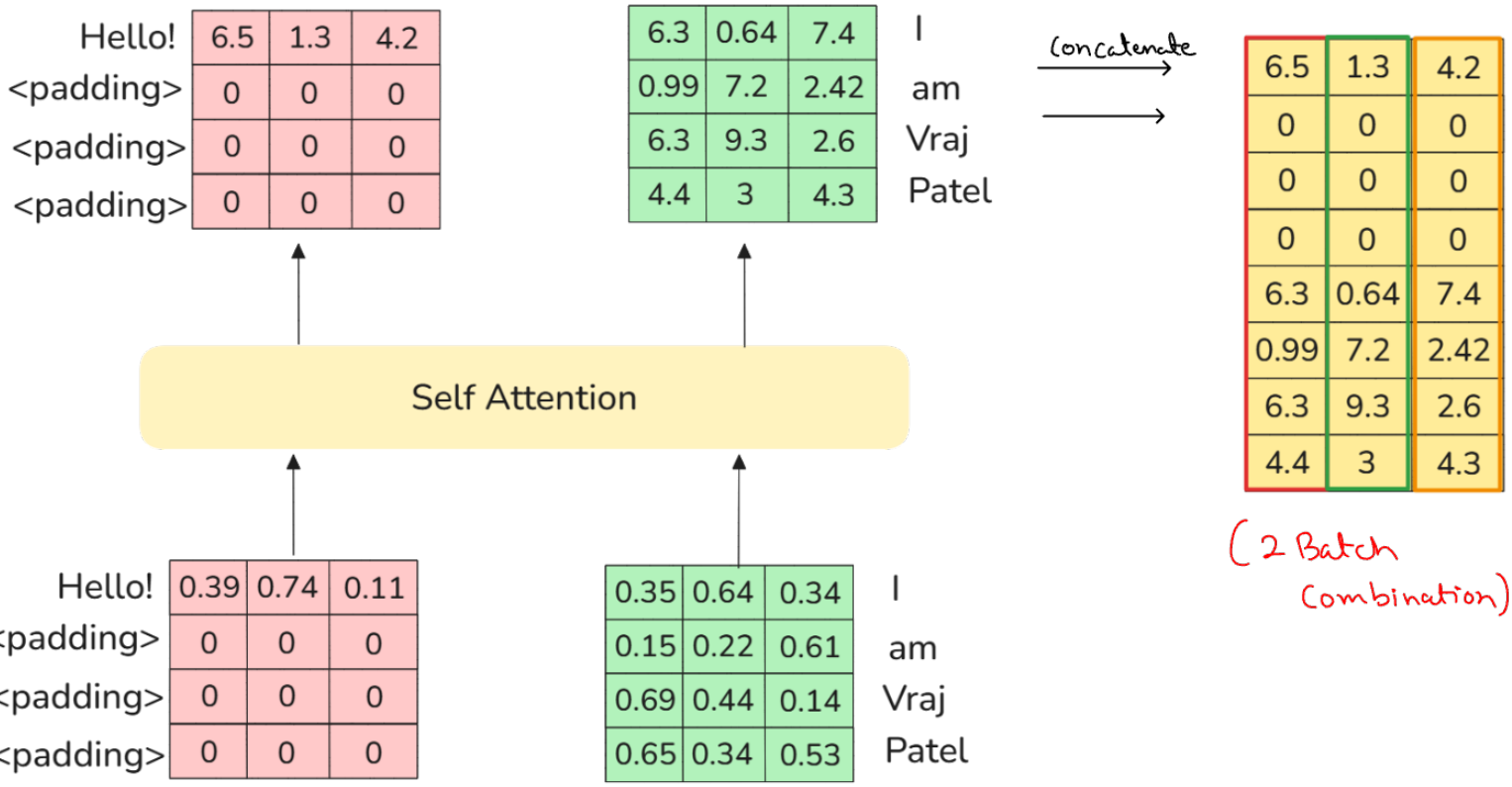


We have converted all the words to embeddings and since it's not necessary for all sentences to have same amount of words, we use padding for short sentences. Once we convert to matrix we can send them.

Now we can send these matrices through a self attention block.

We send matrices as a tensor

tensor dimensions here: (2, 4, 3)
(matrices, row, col)



(2 Batch Combination)

Since we know in batch normalization, we calculate the μ and σ (mean and std) across the Batch, we will not be able to get an accurate μ and σ if the data has 0's.

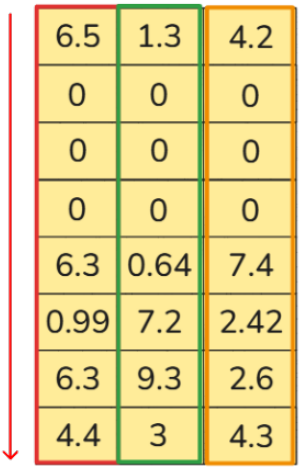
Because of padding we have to deal with 0 which will disturb the μ and σ

red box $= \frac{6.5+0+0+0+6.3+0.99+6.3+4.4}{8}$

↓

This will result in incorrect μ and same with σ.

Therefore, we cannot use Batch Norm with Self attention.



Layer Normalization.

Instead of going through batches vertically, we go through the features (Horizontally)

e_0	e_1	e_2
6.5	1.3	4.2
0	0	0
0	0	0
0	0	0
6.3	0.64	7.4
0.99	7.2	2.42
6.3	9.3	2.6
4.4	3	4.3

γ_1 β_1 γ_2 β_2 γ_3 β_3

Each embedding will have its own gamma (Y) and Beta (β)

Calculating for row 1

Normalizing 6.5

$$\mu = \frac{6.5 + 1.3 + 4.2}{3} = 4.0$$

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2} = 2.12$$

$$N(6.5) = \frac{6.5 - \mu}{\sigma} = \frac{6.5 - 4}{2.12} = 1.18$$

Now multiply z_1 with the γ_1, β_1

$$N(6.5) = 1.18\,(\gamma_1 + \beta_1)$$

Normalizing 1.3

$\mu = 4$

$\sigma = 2.12$

$$N(1.5) = \frac{1.5 - \mu}{\sigma} = \frac{1.5 - 4}{2.12} = \frac{-2.5}{2.12} = -1.18$$

$$N(1.5) = -1.18\,(\gamma_2 + \beta_2)$$

row 3

for 0

$\mu = 0$

$\sigma = 0$

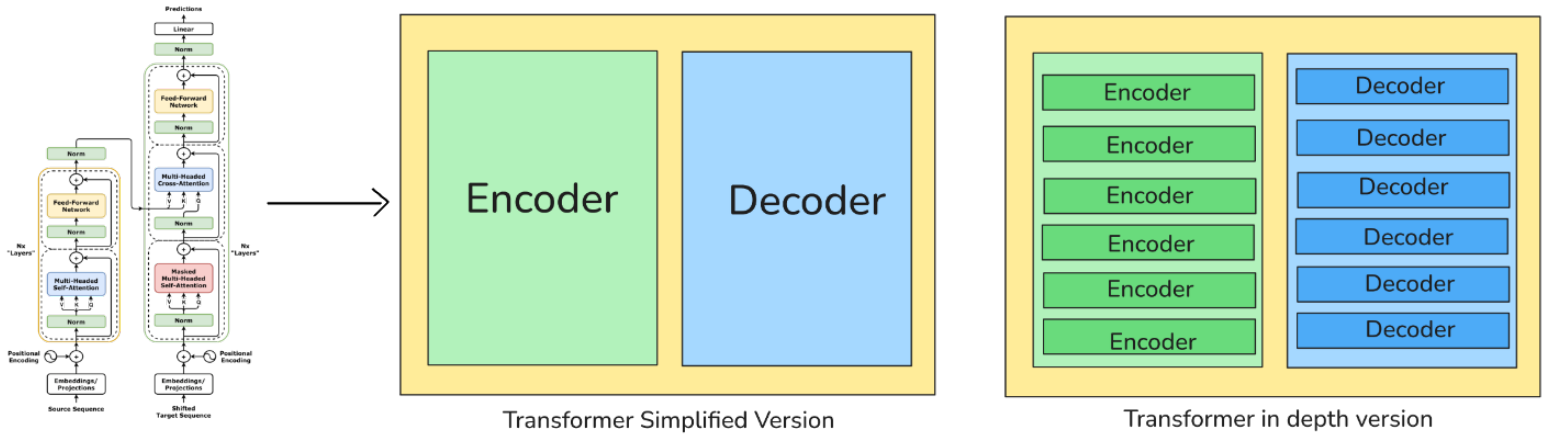
$N(0) = 0$

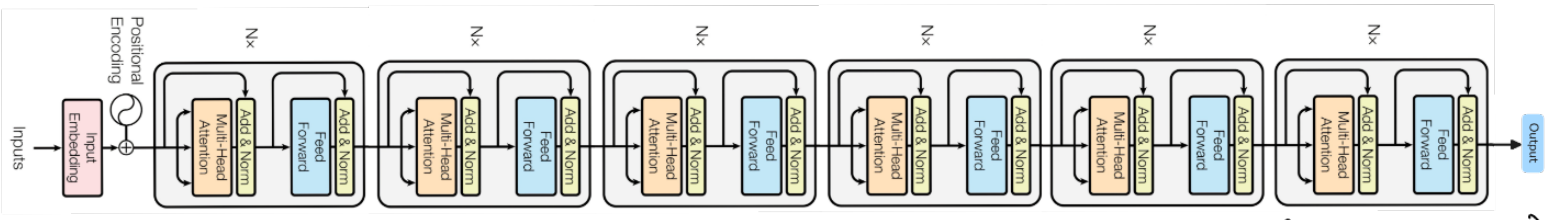
The zeroes will affect themselves and not affect the rest of the data.

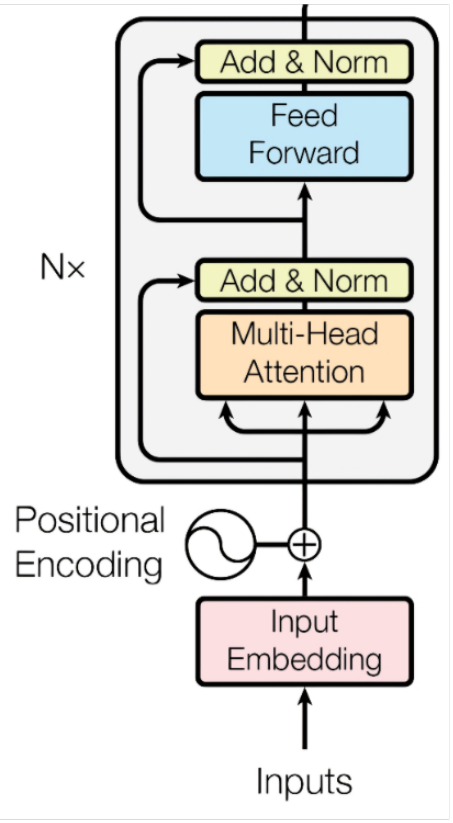
This is the main reason we use Layer normalization instead of Batch normalization.

Transformer Architecture - Encoder

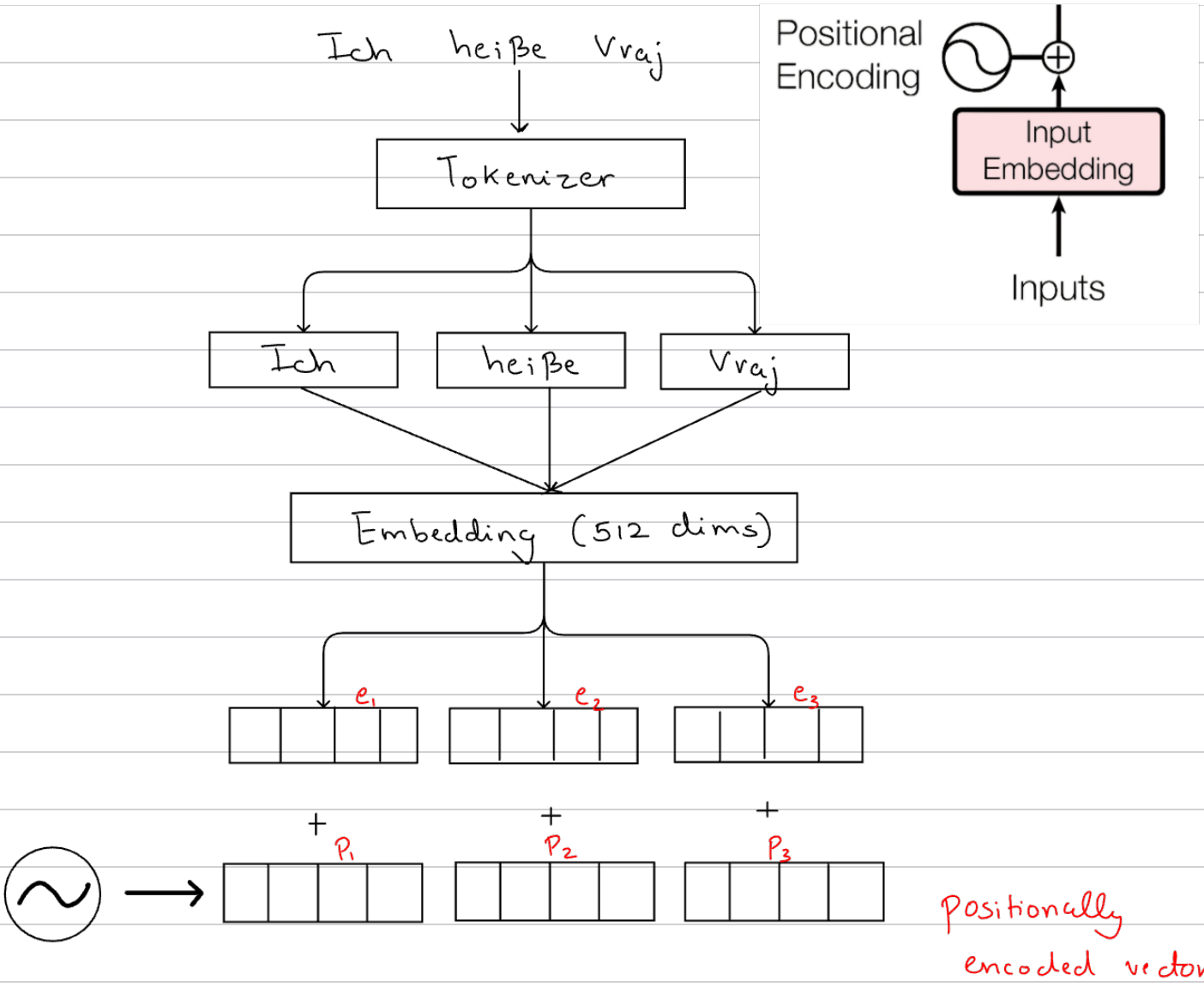
The encoder's job is to take an input sequence and produce a rich context-aware representation for every token. A single encoder block has these components stacked in order.







1. Raw input tokens
2. Convert tokens into embeddings
3. positional encoding to embeddings
4. Generate Q, K, V via linear projection
5. Compute scaled-dot product attention
6. Do this in parallel across all head
7. Concat heads + linear projection (W^0)
8. Add Residual + layer normalization
9. Passing via FNN
10. Add residual + LayerNorm
11. Repeat steps 4 to 10 for N layers
12. Output a context rich representations ready to be sent to decoder.



- The input words are firstly converted to tokens via tokenizer

- The tokens are then converted to embeddings

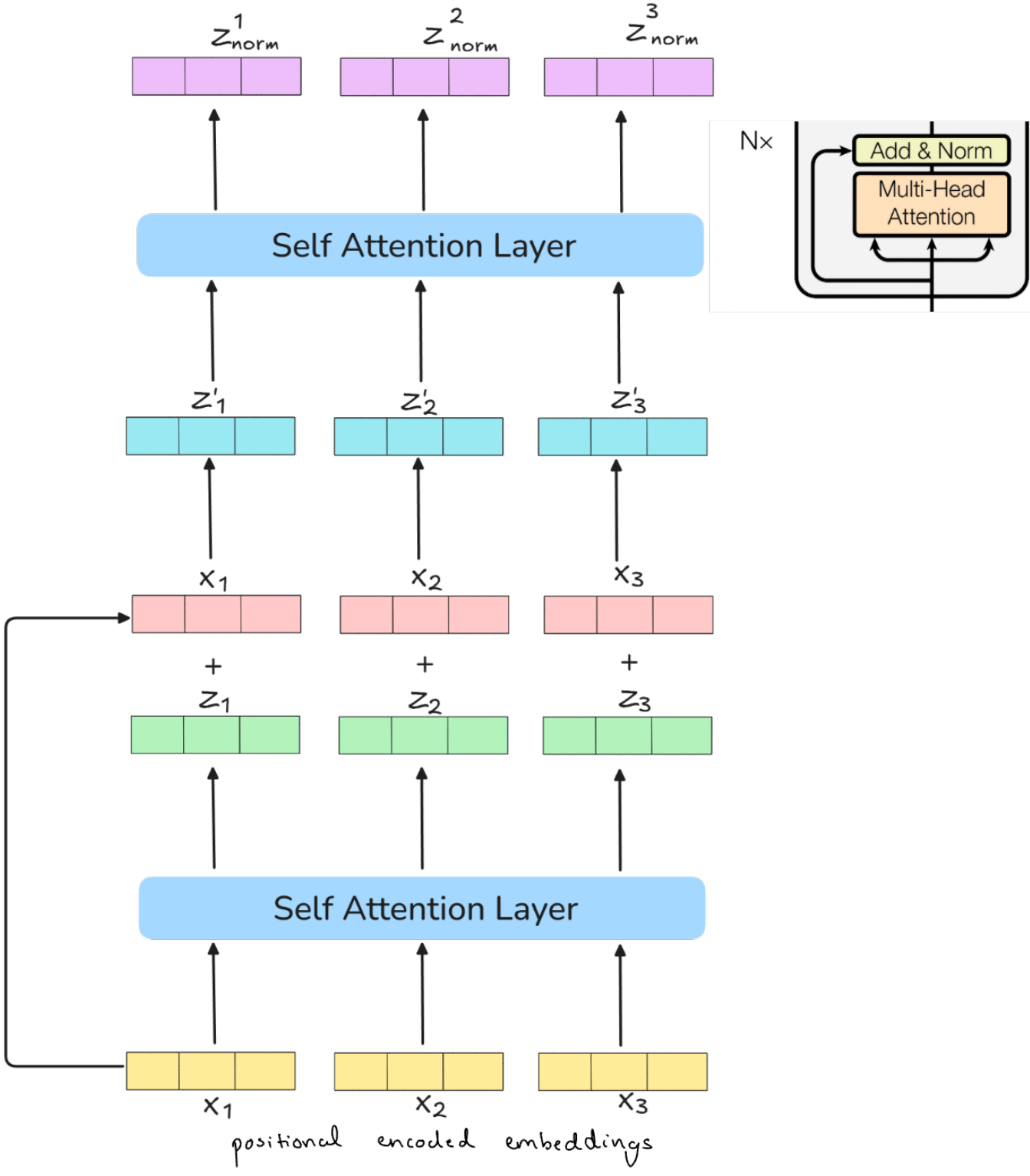
- Embeddings are then converted to positional encoding.

- This result is a single matrix $(n \times d_{model})$ that serves as the actual input to the first encoder block. Everything downstream - attention, FNN etc.- builds on top of this combined representation.

- positional encoded embeddings are firstly passed to self attention layer to output Z embeddings.

- Using residual networks, the positional encoded embeddings (x) are added to the self attention embeddings (z) to give z'

- We now apply layer normalization on z' to get z_{norm} embeddings.



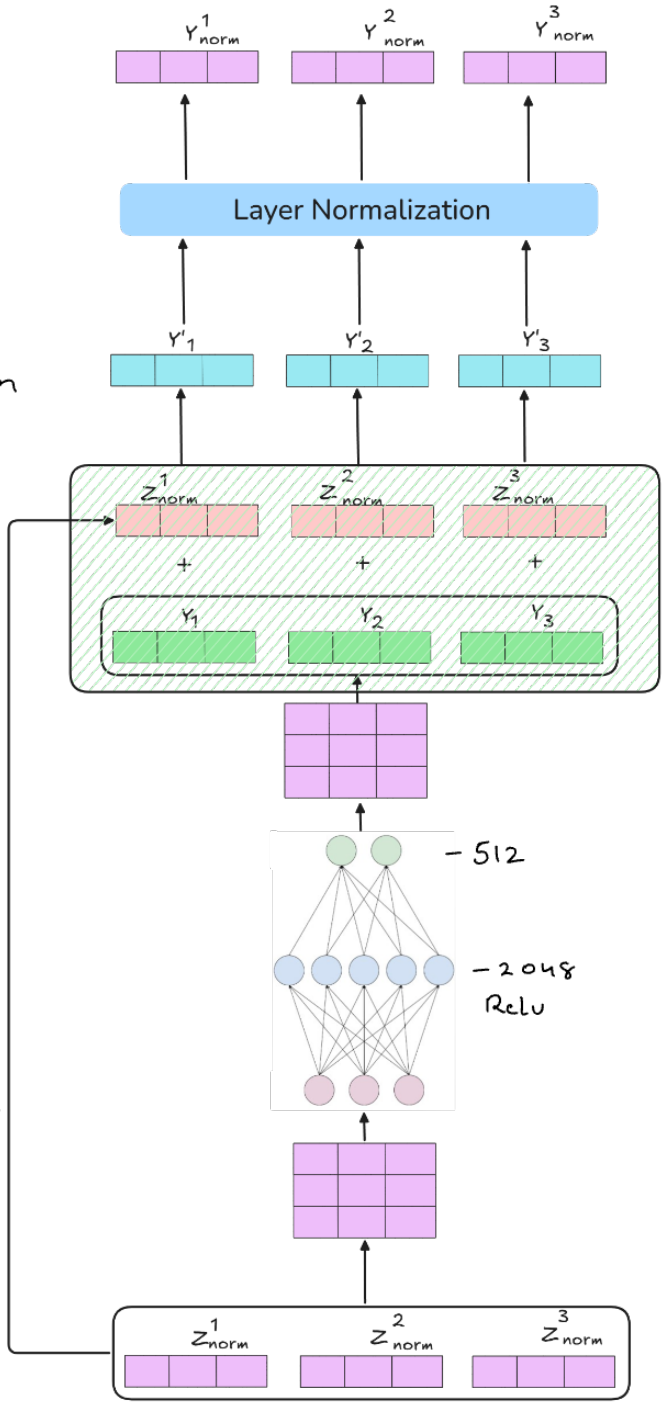
- Each token representation independently passes through two layer MLP.

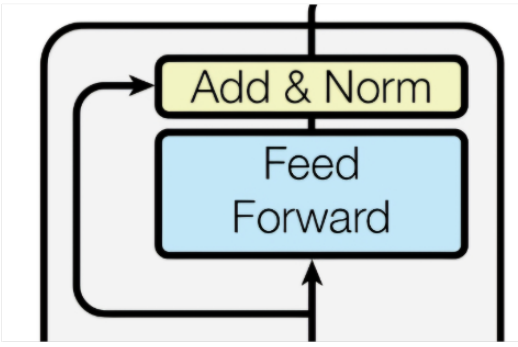
first expanding from $d_{model} = 512$ to $d_{ff} = 2048$ with ReLU activation, then compressing back to 512.

No interaction between tokens here. Attention already figured out the relationship between tokens. FFN now just processes each token on its own. (FNN transforms each token's features individually).

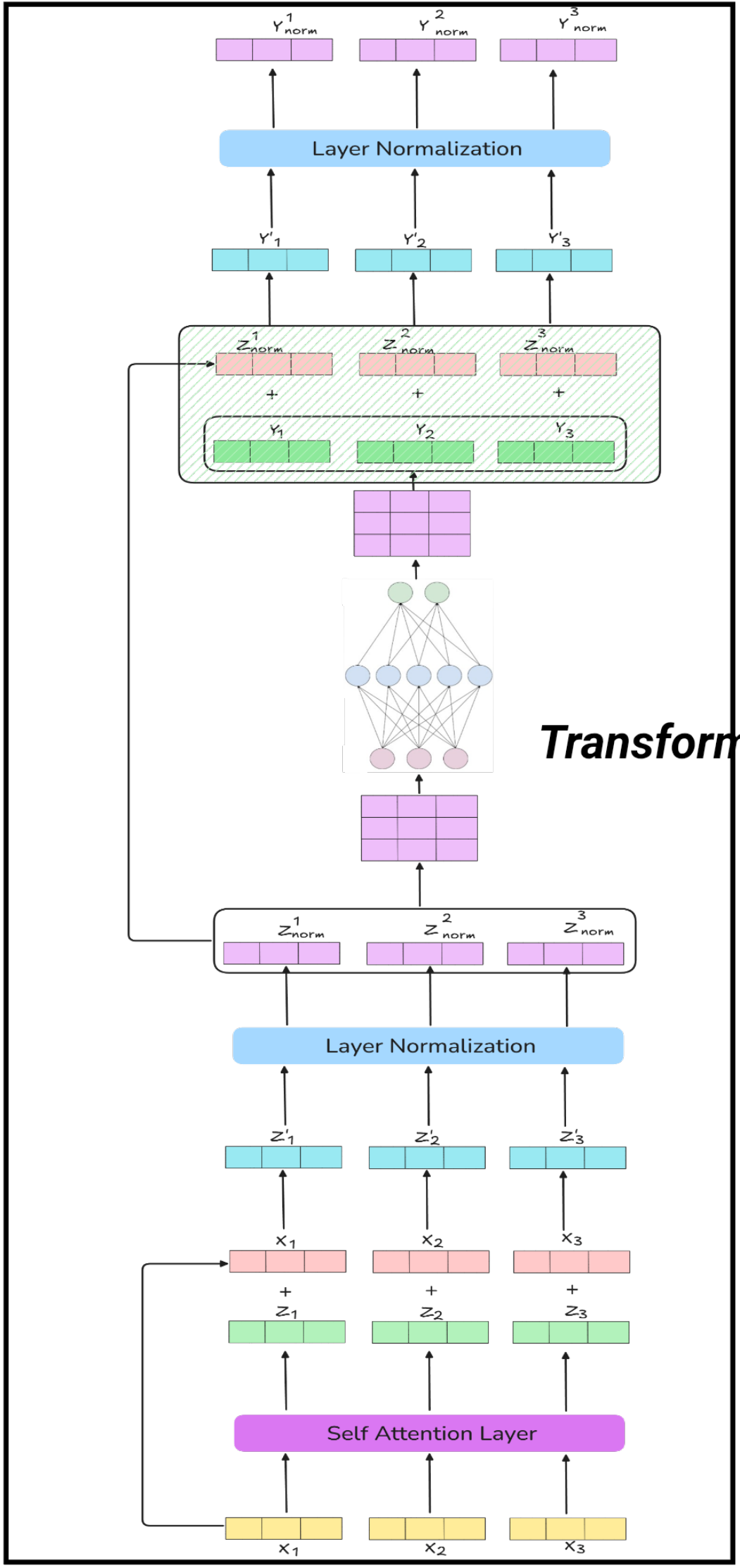
The expansion into higher dimension lets the network learn rich non-linear transformations before compressing back.

- Add & norm : Residual connection adds the input of FNN block (y) back to its output (z_{norm}) followed by layer normalization.





This completes one encoder block. Stack this N=6 times, and the final output is a context rich ($n \times d_{model}$) matrix that gets sent to the decoder.

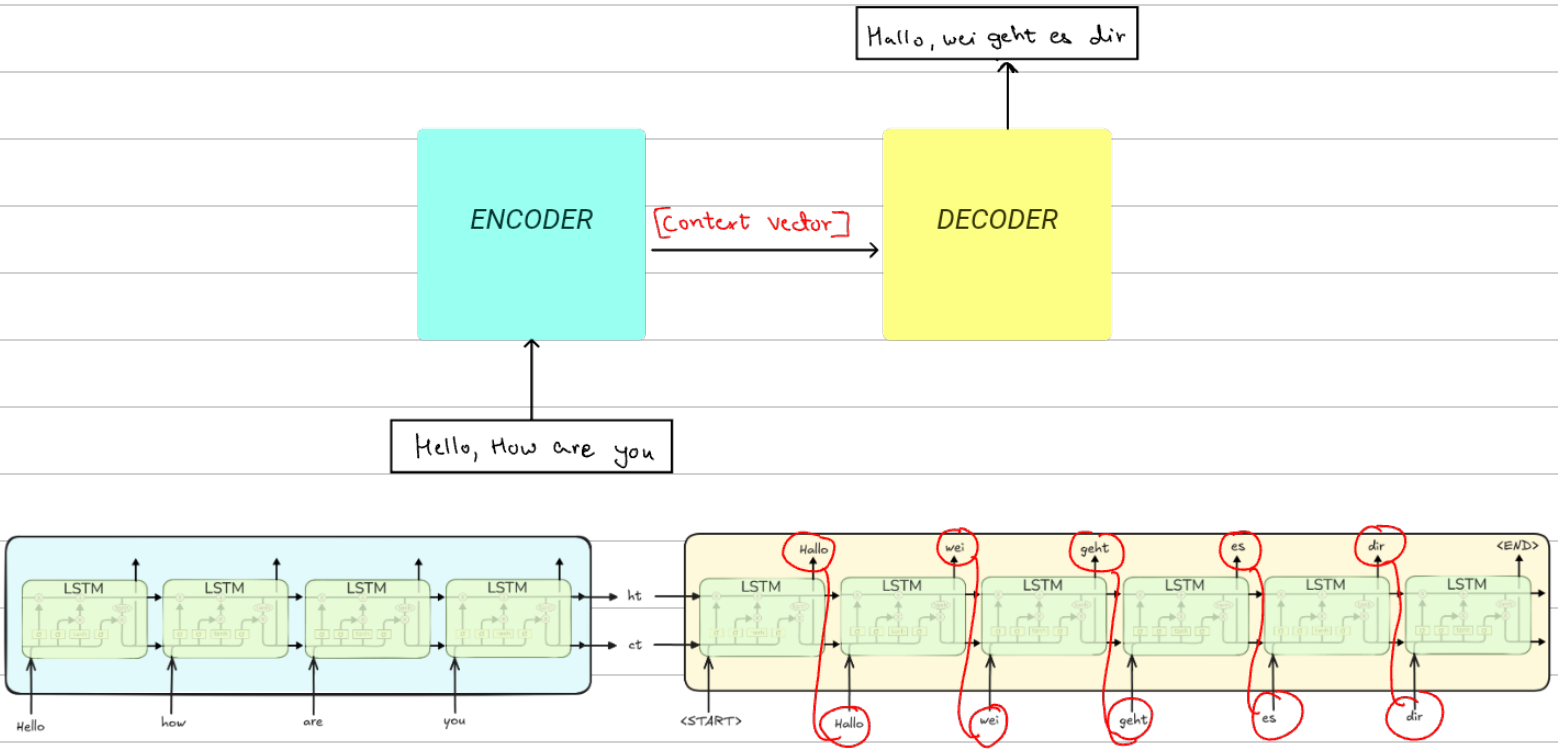

Layer Normalization
Layer Normalization
Self Attention Layer
6 times
Transformer Encoder

Masked Self Attention

The transformer decoder is autoregressive at inference time and non-autoregressive at training time.

Auto regressive : A model is autoregressive when it generates output one step at a time, where each step's input depends on the output of previous step. So the model literally "regresses on its own prior outputs" - it uses what it already generated to decide what to generate next.

lets start this by autoregressive in vanilla Seq2seq



Auto regressive

Here the problem is that it is inherently sequential. We cannot parallize it because step t needs the output of step t-1

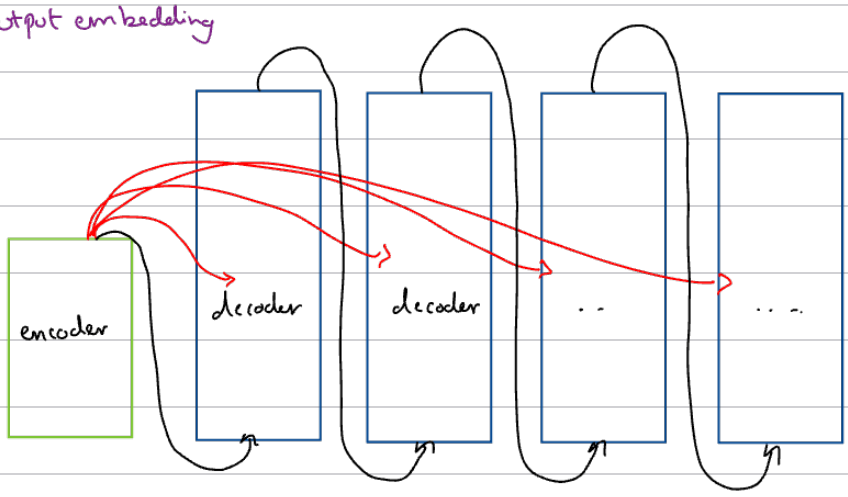
So, in transformer decoder, while testing/predicting, it is autoregressive. We have no option

training

Now, during training, we have transformer:

Sentence = "how are you"

inputs: i_1 (in decoder): to multihead attention
i_2: to output embedding



each timestep gets one input from encoder and another from previous timestep's output

But since this is while training, we also have to make sure that the model predicts the correct token in the next timestep.

So, if our decoder predicts the wrong token, we still provide the next timestep t+1 the correct input from the training data. This is simply "teacher forcing".

So, we can notice that no matter if our decoder predicts the correct token or not, we will be providing it the correct input from the training data to the next decoder timestep. Therefore, in reality - no timestep is dependent on its previous t-1 timestep's output. It simply depends on encoder's output and the data's output token.

Therefore we can induce parallization here.
There is no point of sending each token sequencially because we can easily induce parallelism.

So, now we try parallel

We know from self attention that : for each token, it computes a weighted sum over all tokens based on learned attention scores. So when computing the new representation for the sentence "The train arrives soon":

0.2 The + 0.3 train + 0.4 arrives + 0.1 soon

0.1 The + 0.5 train + 0.35 arrives + 0.05 soon

0.3 The + 0.4 train + 0.1 arrives + 0.2 soon

0.1 The + 0.3 train + 0.5 arrives + 0.1 soon

The above equations simply shows the similarity between each other.

ex: $e^{new}_{arrives} = \underbrace{0.3 e_{the}}_{\text{Similarity between "the" \& "arrives"}} + \underbrace{0.4 e_{train}}_{\text{Similarity between "train" \& "arrives"}} + \underbrace{0.1 e_{arrives}}_{\text{Similarity between "arrives" \& "arrives"}} + \underbrace{0.2 e_{soon}}_{\text{Similarity between "soon" \& "arrives"}}$

The problem : The model when processing "the", is directly looking at "train", "arrives", and "soon" - the very tokens its supposed to predict in the future. This is data leakage. The model can simply copy answers from the future instead of learning actual language patterns.

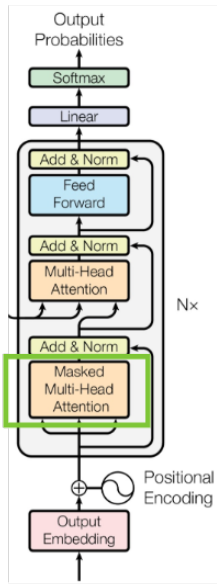
Then why not just use auto regressive Trainings?

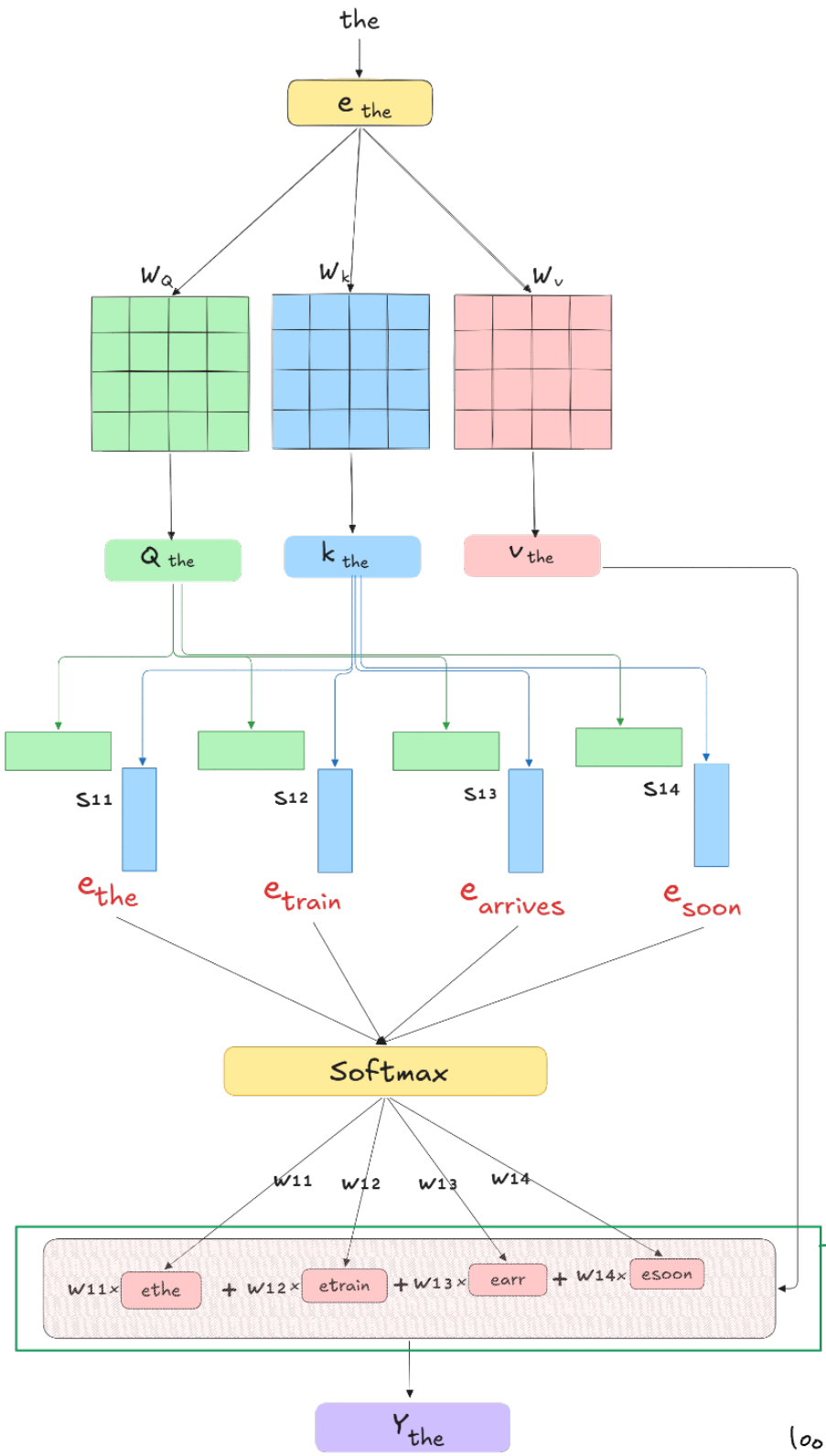
→ Yes, autoregressive training avoids this problem entirely because future tokens literally don't exist yet at each timestep. But sequential training is extremely slow. We need parallelism over TB's of data.

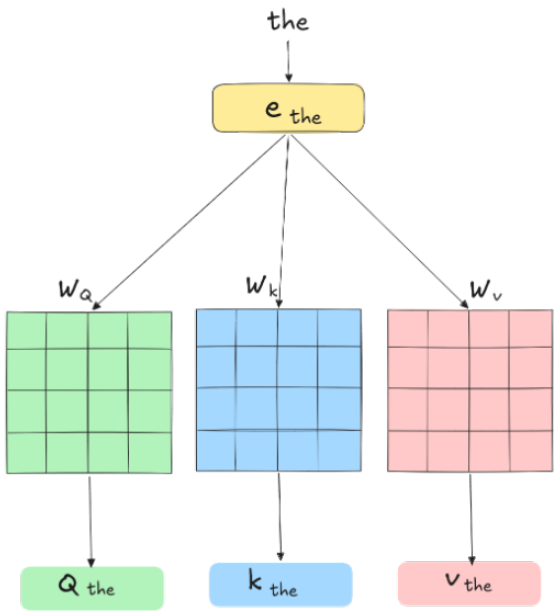
So, we want the casual property of autoregressive generation combined with computational efficiency of parallel processing

Masked Self Attention – Solution.

The problem is the root of self attention. lets fix it!

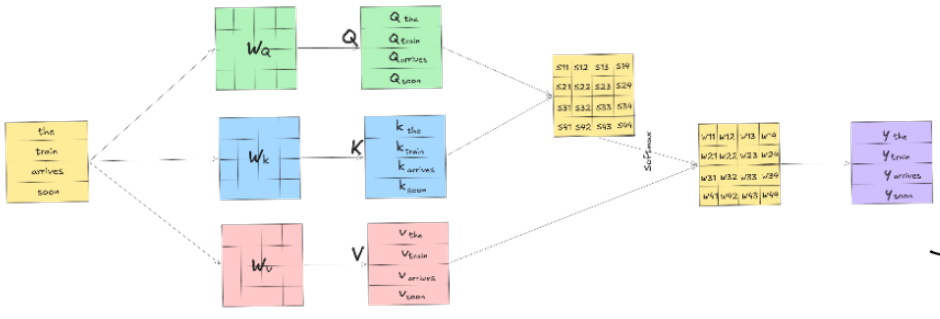






This is our usual flow.

Here's the main culprit. Even though we have "the" token so far, we are still looking at the next future tokens.



We introduce a mask that will make all the future values '0' so that the data does not leak.

generally:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right).V$$

Now, we introduce a mask

$$\text{Masked Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right).V$$

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$-\infty$ because when softmax applied, those values becomes 0

Notice, if we concatenate matrix M with the S matrix:

S11	S12	S13	S14
S21	S22	S23	S24
S31	S32	S33	S34
S41	S42	S43	S44

$$+ \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

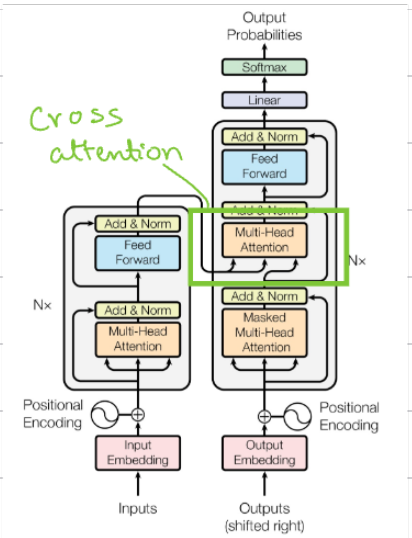
$$\begin{bmatrix} S_{11} & 0 & 0 & 0 \\ S_{21} & S_{22} & 0 & 0 \\ S_{31} & S_{32} & S_{33} & 0 \\ S_{41} & S_{42} & S_{43} & S_{44} \end{bmatrix}$$

No data leakage! because the future tokens are 0.

The mask is just a matrix. It's a fixed pre-computed tensor.
So now during training: non-autoregressive (parallel), no data leakage

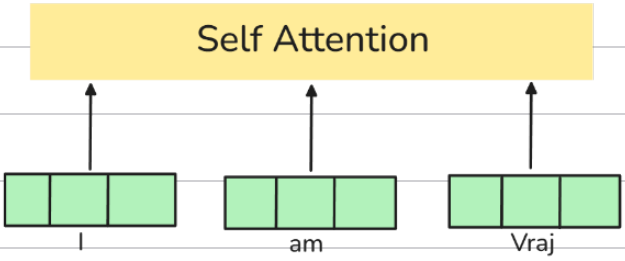
Masked self attention is the mechanism that bridges these two models. It allows the transformer decoder to train while preserving the casual constraint that each token can only depend on past tokens.

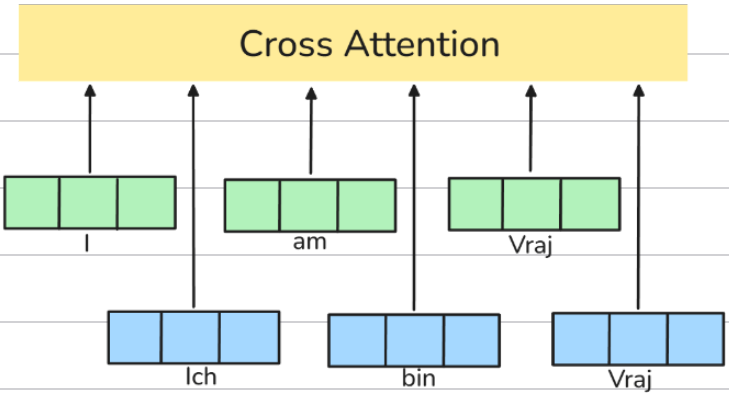
Cross Attention



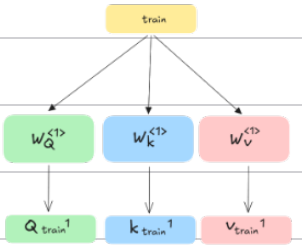
Cross attention is a mechanism used in transformer architectures, particularly in tasks involving sequence to sequence data like translation or summarization. It allows a model to focus on different parts of an input sequence when generating an output sequence.

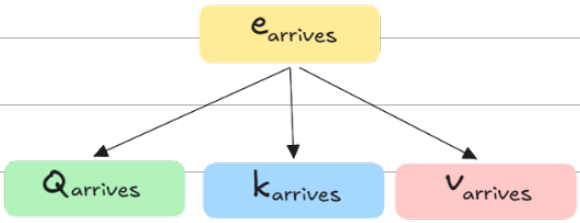
Simply, self attention is to find out similarity with all the other tokens in the sequence while on the other hand, cross self attention is to find out similarity with all other tokens from two different sequence.

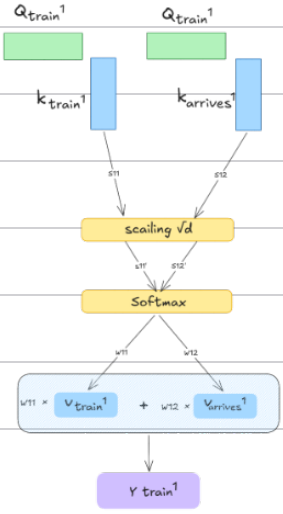




Vanilla Self Attention



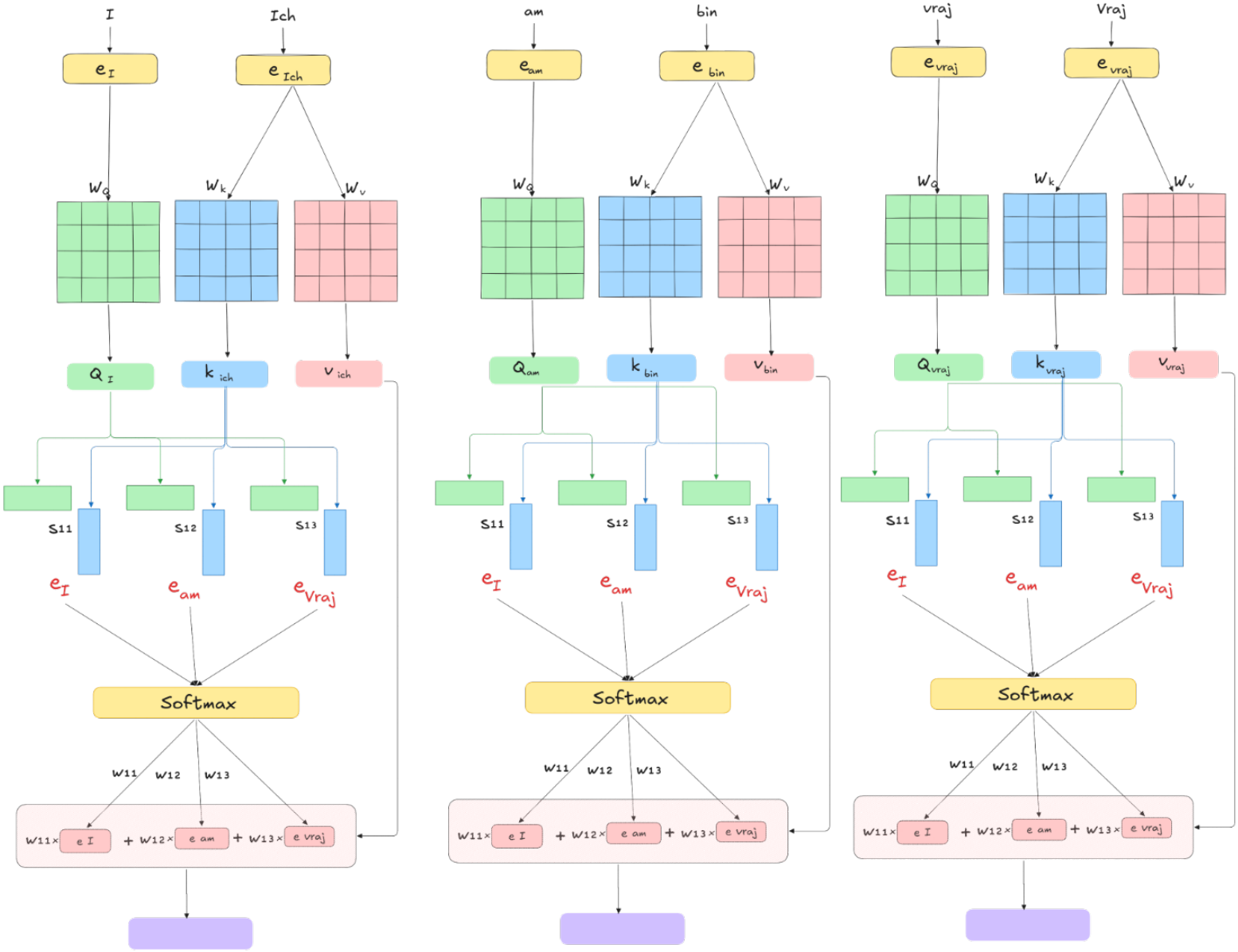


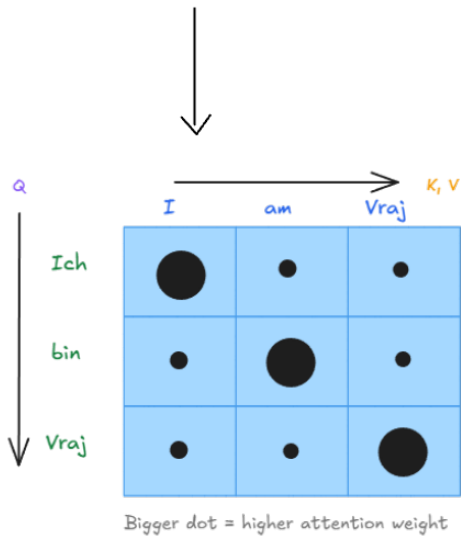


In vanilla self attention, the tokens in the sequence are compared with itself and provides an output

In cross attention, :

Query : Comes from decoder previous output
Key : } Comes from encoder output
Value :





Cross-Attention in Transformers

Cross-attention allows the **decoder** to attend to the **encoder's output**, connecting the two halves of the Transformer architecture.

How It Differs from Self-Attention

In self-attention, Q, K, V all come from the same sequence. In cross-attention:

- **Q ← decoder** (current target representation)
- **K and V ← encoder's final output**

Decoder asks: *"Given what I've generated (Q), which parts of the input (K, V) should I attend to?"*

	Self-Attention	Cross-Attention
Q	Same sequence	Decoder
K	Same sequence	Encoder output
V	Same sequence	Encoder output

The Computation

Mechanically **identical** to self-attention — only difference is ***where*** Q, K, V originate: $\texttt{Attention}(Q,K,V) = \texttt{softmax}(QK^T / \sqrt{d_k}) \cdot V$

Weight Matrices in Detail

Each cross-attention sub-layer has its own learned projection matrices:

- $Q = H_{dec} \cdot W_Q$ where H_{dec} is the decoder's hidden state from the previous sub-layer
- $K = H_{enc} \cdot W_K$ and $V = H_{enc} \cdot W_V$ where H_{enc} is the encoder's final output

W_Q, W_K, W_V are **not shared** with the self-attention sub-layers — each cross-attention layer learns its own set of projections.

Multi-Head Cross-Attention

Just like self-attention, cross-attention uses **multiple heads**. Each head independently projects Q (from decoder) and K, V (from encoder) into $d_k = d_{model}/h$ dimensions, computes attention, and the results are concatenated and linearly projected via W_O. Different heads can learn different alignment patterns — e.g., one head may align nouns, another may align positional structure.

Position in the Decoder Block

1. **Masked Self-Attention** — attends to previously generated tokens (causal mask)
2. **Cross-Attention** — attends to encoder output (*no mask* — decoder can see the full source)
3. **Feed-Forward Network** — position-wise MLP applied independently to each token

Encoder output (K, V) is computed once and reused across all decoder layers and time steps — making inference efficient.

Why It Matters

Without cross-attention, the decoder has **no way to access the source input**. In translation, it learns *soft alignments* between source and target words — like Bahdanau attention but parallelized. Unlike hard alignment (one-to-one), soft alignment lets each target token attend to *all* source positions with different weights.

Dimension Note

Self-attention: $T \times T$ (square). Cross-attention: $T_{target} \times T_{source}$ (rectangular). This also means cross-attention doesn't need a causal mask since the encoder sequence is fully available.

Where Cross-Attention Exists

- **Encoder-Decoder (Transformer, T5, BART): ✓** Has cross-attention
- **Decoder-Only (GPT): ✗** No encoder to attend to
- **Encoder-Only (BERT): ✗** No decoder at all

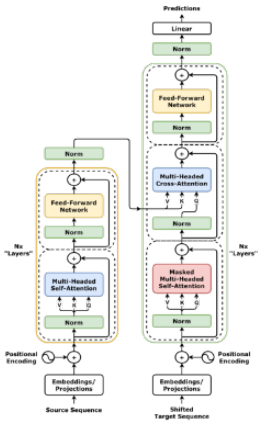
Cross-Attention Beyond NLP

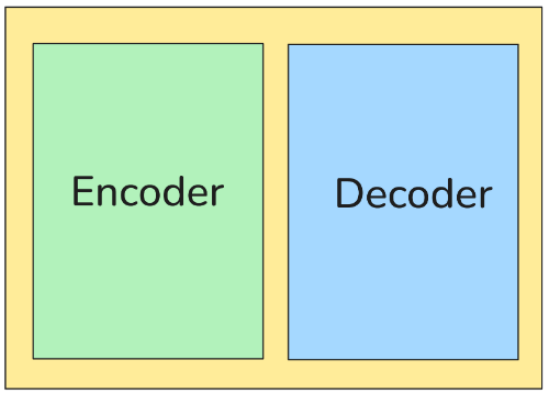
Cross-attention is not limited to text. In **Stable Diffusion**, the U-Net uses cross-attention where Q comes from the image latent and K, V come from the text embedding — this is how the model conditions image generation on a text prompt. In **Vision Transformers (ViT)** with detection heads, cross-attention lets object queries attend to image patch features. The pattern is general: any time one modality/stream needs to condition on another, cross-attention is the standard mechanism.

Cross-attention is the bridge between encoder and decoder — and more broadly, between any two representations that need to interact.

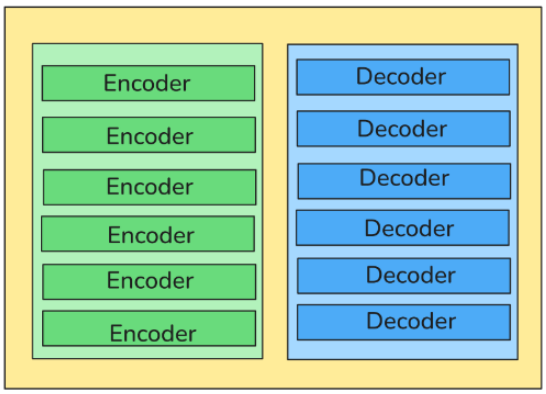
Transformer Decoder Architecture While Training

Decoder behaves like non-auto regressor during training



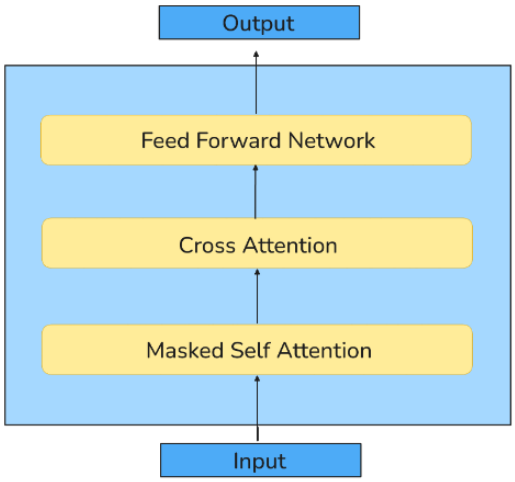


Transformer Simplified Version

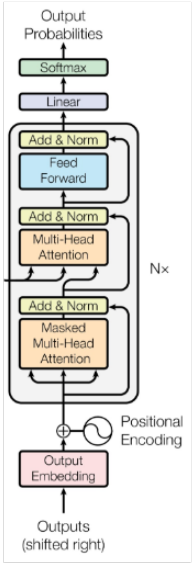


Transformer in depth version

Simplified Decoder



OR

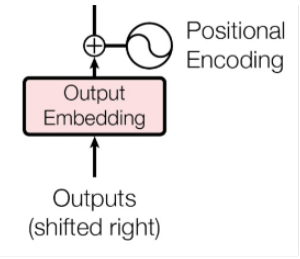


Steps:

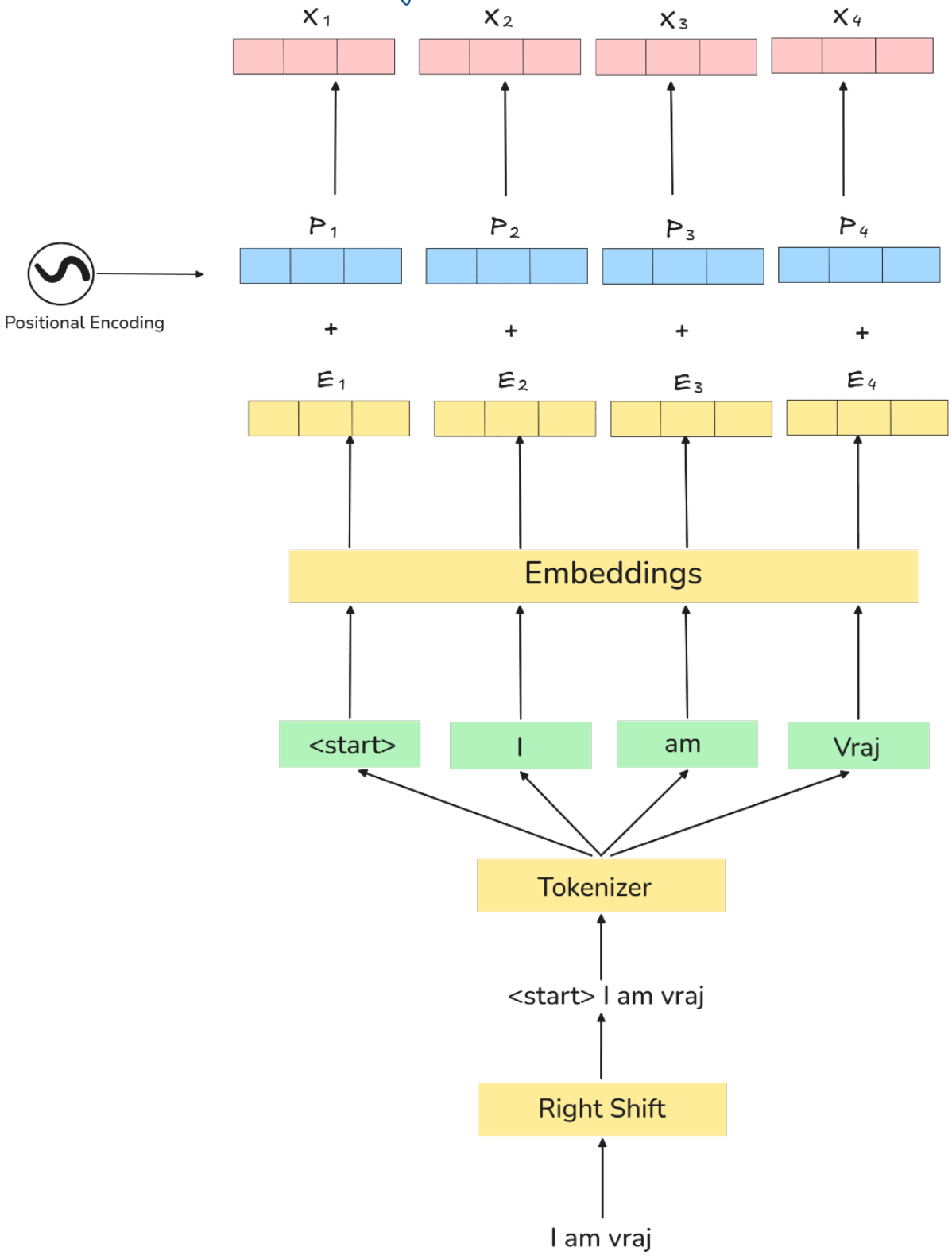
1. Target sequence shifted to right by one position
2. Tokens to embeddings
3. Positional encoding
4. Generate Q, K, V via linear projections
5. Scaled dot product with masked self attention
6. Do this in parallel across h heads
7. Concat head + linear projection (W^o)
8. Add residual + layerNorm (2nd)
9. Q from decoder, K, V from encoder final output → Cross attention
10. Residual + LayerNorm
11. FNN
12. Repeat 4 - 12 steps
13. Linear layer → project to vocabulary size
14. Softmax
15. Compute loss

During training

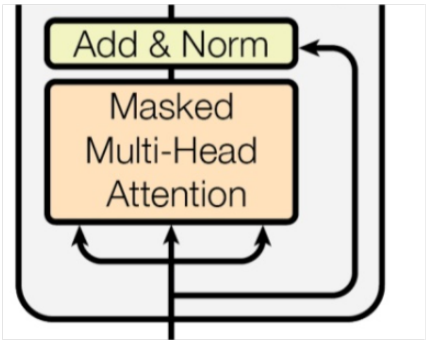
Phase ① :
i. Shifting
ii. Tokenization
iii. Embedding
iv. Positional Encoding

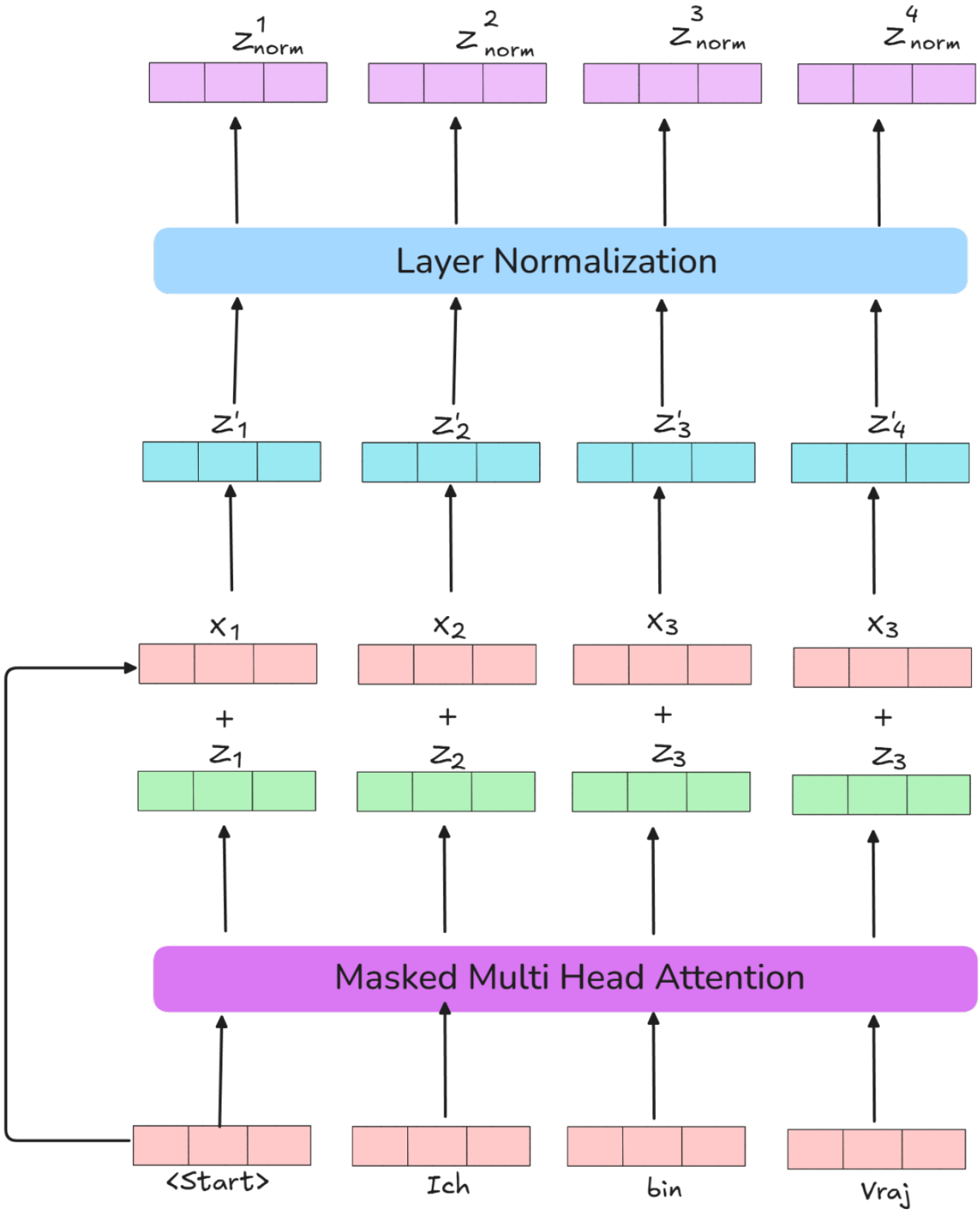


Sentence: I am Vraj
Ich bin Vraj



Phase 2: Masked Multihead Attention
+
Add & layer Norm.





Note: This is the target scentence : German translation

Phase (3) Cross Attention

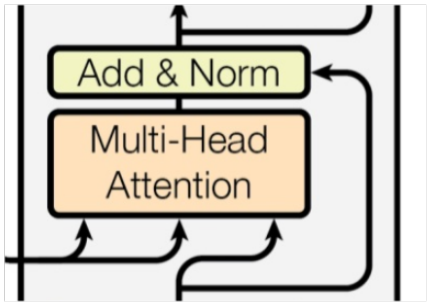
2 input from encoder output

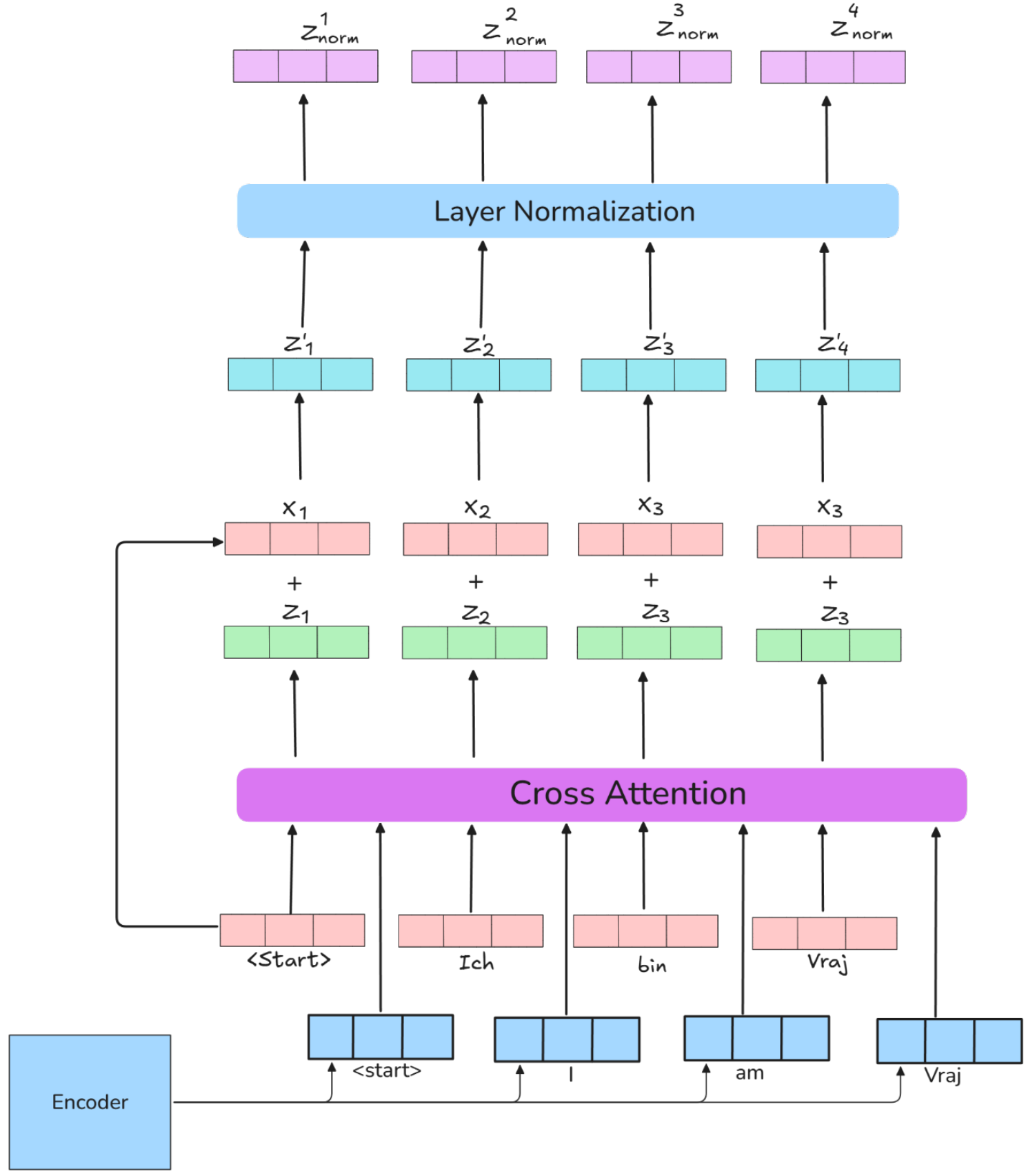
+

1 input from Masked Attention block

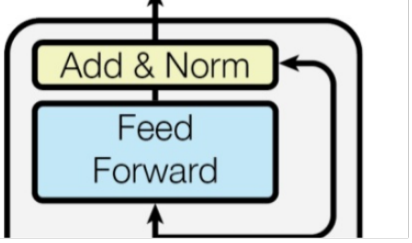
+

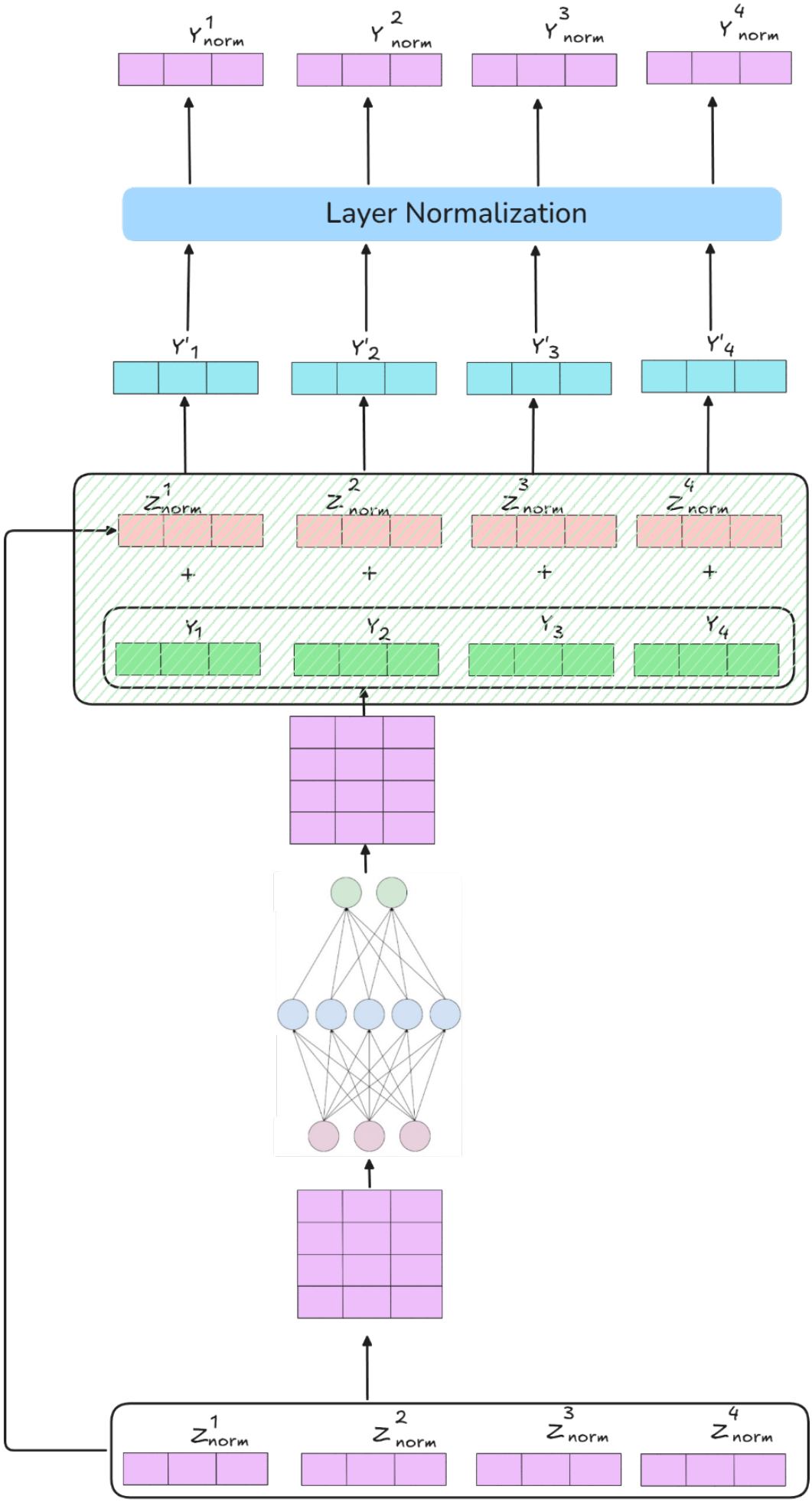
Add & Norm



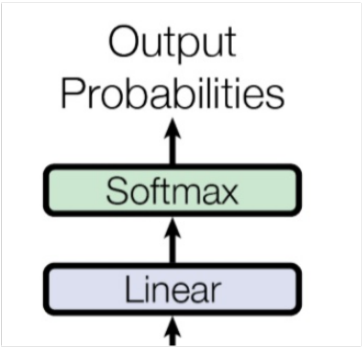


Phase 4: FFN


Add & Norm
Feed Forward


Y^1_{norm}
Y^2_{norm}
Y^3_{norm}
Y^4_{norm}
Layer Normalization
Y'_1
Y'_2
Y'_3
Y'_4
Z^1_{norm}
Z^2_{norm}
Z^3_{norm}
Z^4_{norm}
+
+
+
+
Y_1
Y_2
Y_3
Y_4
Z^1_{norm}
Z^2_{norm}
Z^3_{norm}
Z^4_{norm}

Phase 5: Final Output layer



Output Probabilities

Softmax

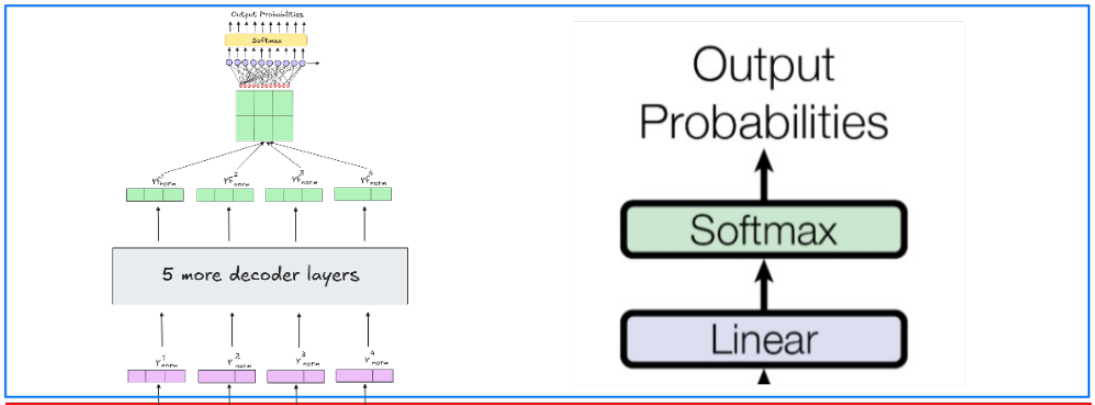
Neurons decided from total number of unique words in your target sentence

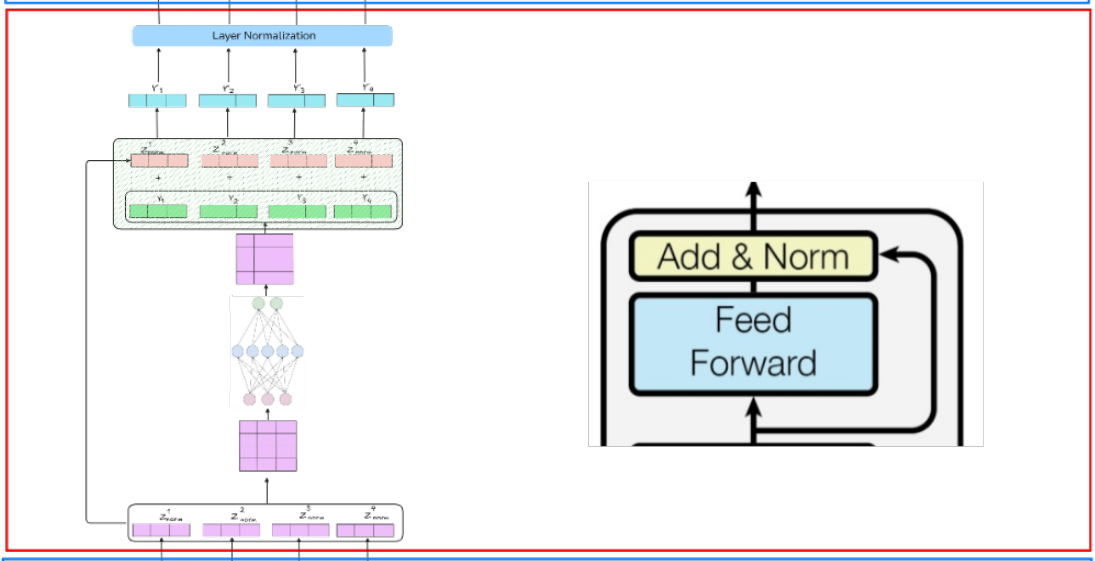
$\{V\}$

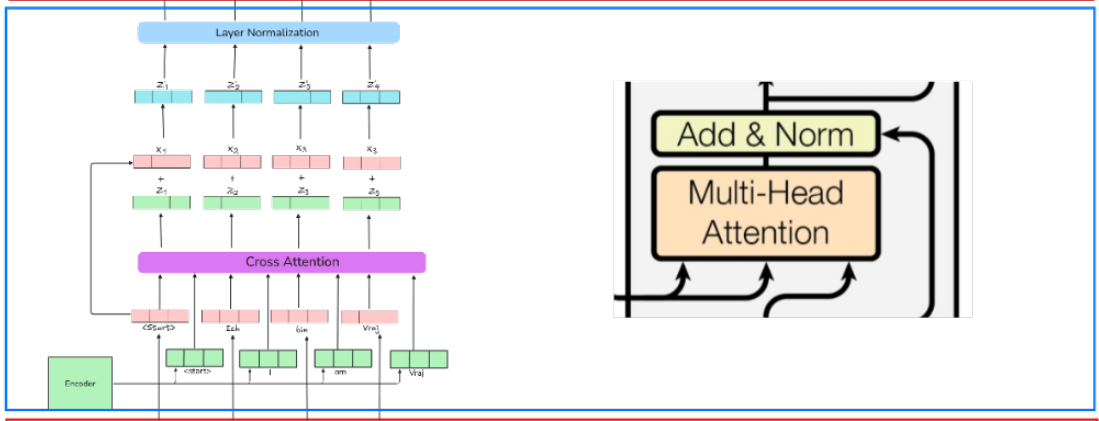
Yf^{1}_{norm} Yf^{2}_{norm} Yf^{3}_{norm} Yf^{4}_{norm}

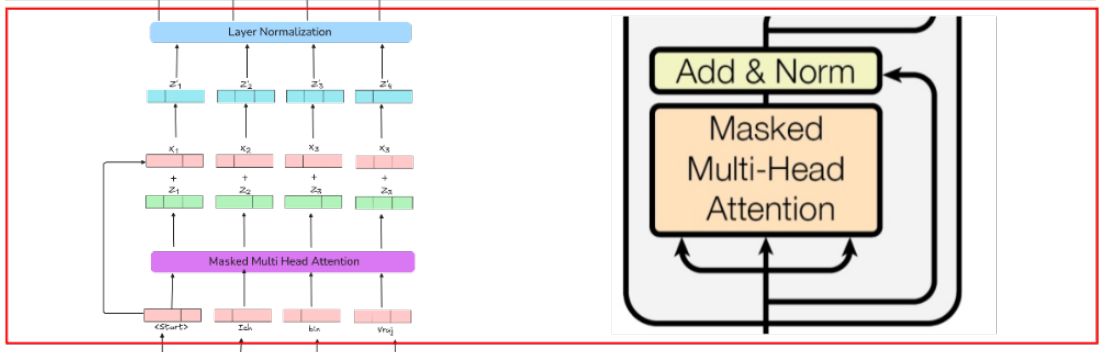
5 more decoder layers

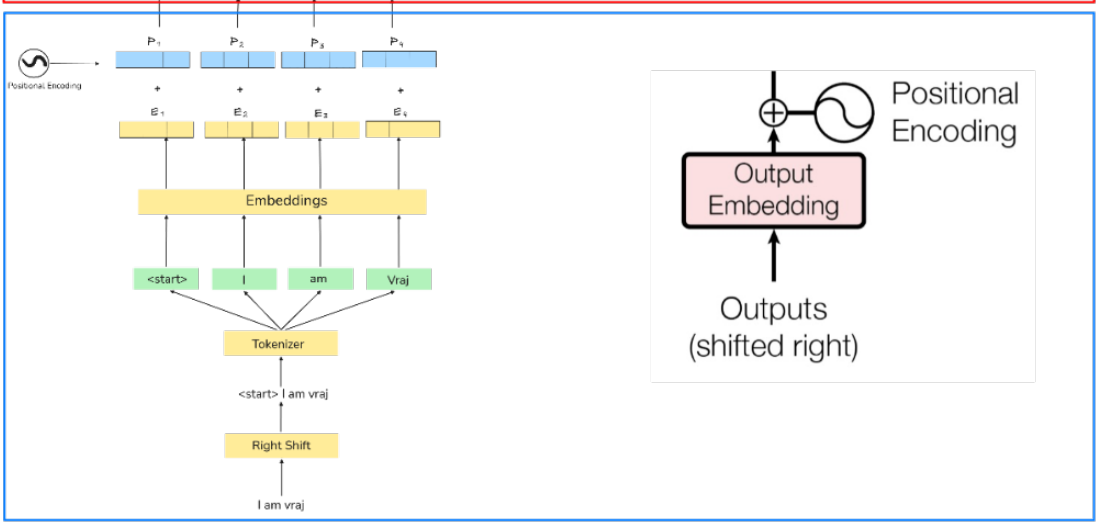
Y^{1}_{norm} Y^{2}_{norm} Y^{3}_{norm} Y^{4}_{norm}


Output Probabilities
Softmax
Linear
5 more decoder layers


Layer Normalization
Add & Norm
Feed Forward


Layer Normalization
Cross Attention
Add & Norm
Multi-Head Attention
Encoder


Layer Normalization
Masked Multi Head Attention
Add & Norm
Masked Multi-Head Attention


Positional Encoding
Embeddings
<start>
I
am
Vraj
Tokenizer
<start> I am vraj
Right Shift
I am vraj
Positional Encoding
Output Embedding
Outputs (shifted right)

Transformer Architecture during Inference (Prediction)

Decoder behaves like auto regressor during prediction

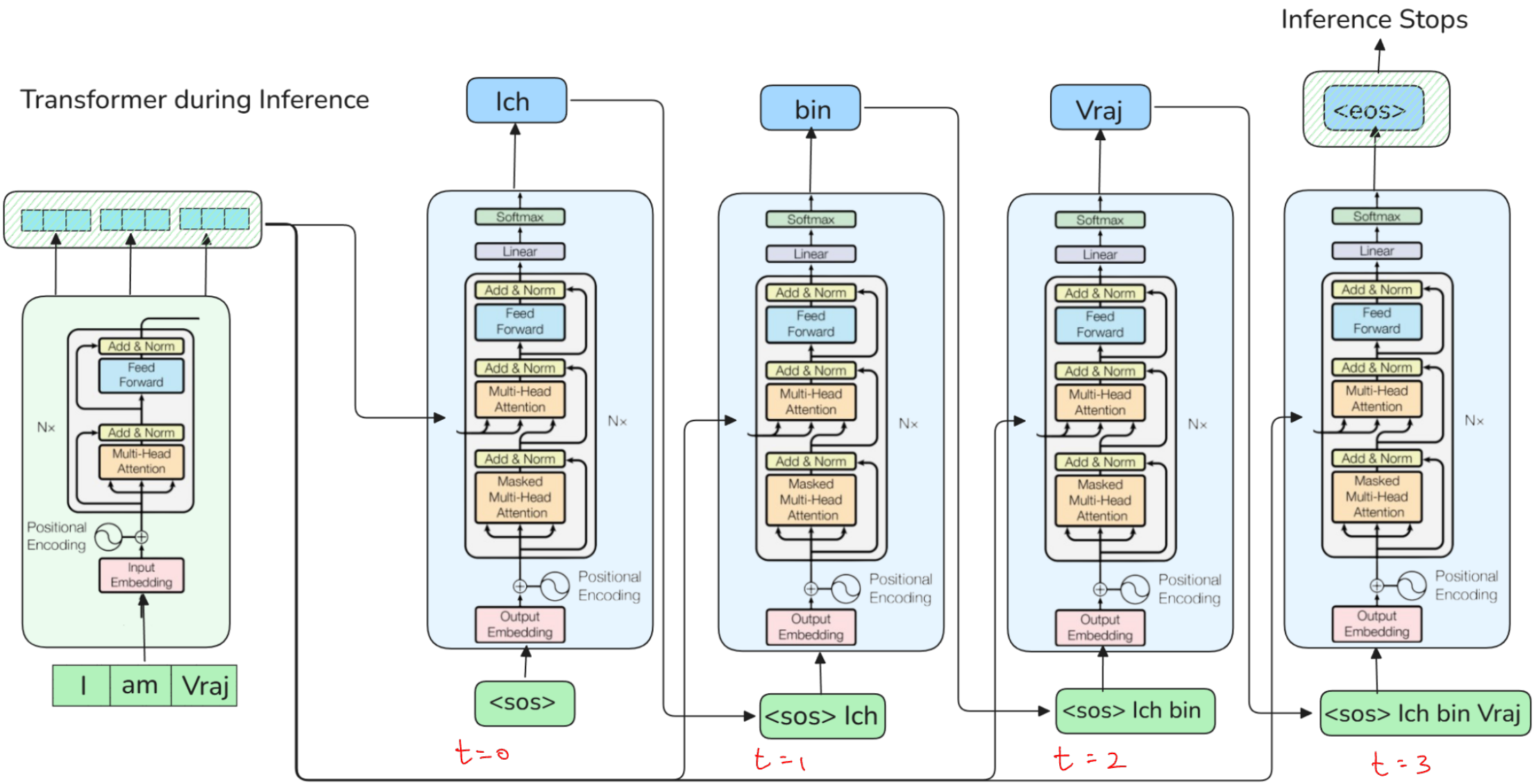
In transformer, decoder is the only part that behaves differently during training and Inference.
decoder is auto regressive during Inference (with timesteps)
decoder is non auto regressive during Training (parallel)

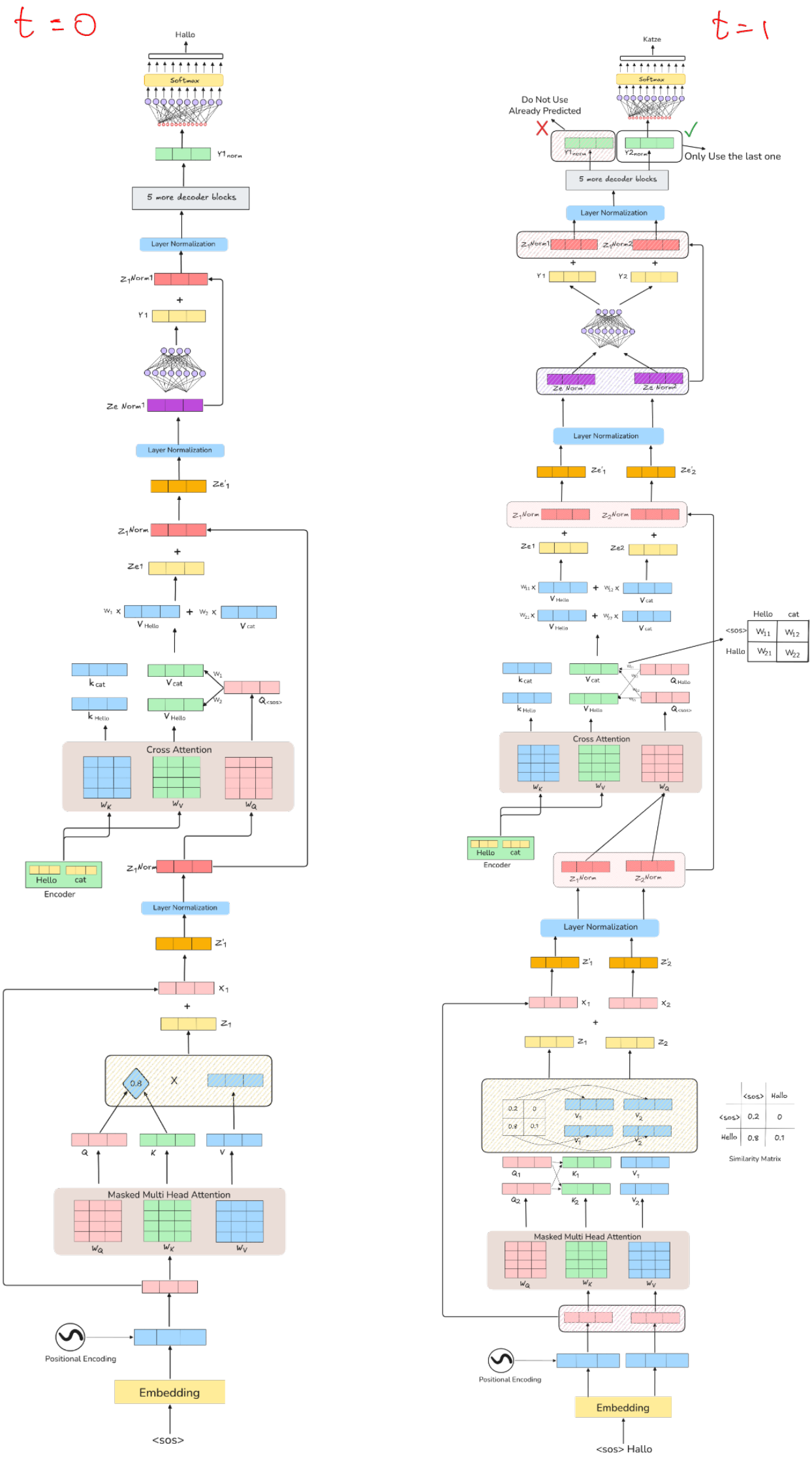
Difference from decoder training :
during Inference,

① Masked attention still applies : Yes, even though we technically dont have future tokens to peek at, the casual mask is still architecturally present. In practice with KV caching, its handled implicitly, The new tokens query just attends against all cached keys which are only past tokens anyway.
In summary, we still use masked attention even though it is not helpful. It is because we have to make predictions the same as the training was done. We cannot change architecture : bad results.

② Timestep wise generation : Step t : feed all tokens generated so far → run through decoder → get output → append new token → repeat. The decoder sees a growing sequence each time.

③ Only the last vector goes to final linear layer :
During training, the linear + Softmax is applied to all position vectors since you compute loss at every position.
During Inference, only the last vector is passed through - its the only one predicting something new. The rest were already predicted by the model previously.


Transformer during Inference
Inference Stops
I
am
Vraj
Input Embedding
Positional Encoding
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Add & Norm
Masked Multi-Head Attention
Output Embedding
Linear
Softmax
<sos>
Ich
<sos> Ich
bin
<sos> Ich bin
Vraj
<sos> Ich bin Vraj
<eos>
t=0
t=1
t=2
t=3


t = 0
Hallo
Softmax
Y1 norm
5 more decoder blocks
Layer Normalization
Z1Norm1
Y1
Ze Norm1
Layer Normalization
Ze'1
Z1Norm
Ze1
W1 x V Hello + W2 x V cat
k cat
V cat
k Hello
V Hello
Q<sos>
Cross Attention
WK
WV
WQ
Hello cat
Encoder
Z1Norm
Layer Normalization
Z'1
X1
Z1
0.8
X
Q
K
V
Masked Multi Head Attention
WQ
WK
WV
Positional Encoding
Embedding
<sos>
t = 1
Katze
Softmax
Do Not Use
Already Predicted
Only Use the last one
Y1norm
Y2norm
5 more decoder blocks
Layer Normalization
Z1Norm1
Z1Norm2
Y1
Y2
Ze Norm1
Ze Norm2
Layer Normalization
Ze'1
Ze'2
Z1Norm
Z2Norm
Ze1
Ze2
V Hello
V cat
k cat
V cat
Q Hallo
k Hello
V Hello
Q<sos>
Hello cat
<sos> W11 W12
Hallo W21 W22
Cross Attention
WK
WV
WQ
Hello cat
Encoder
Z1Norm
Z2Norm
Layer Normalization
Z'1
Z'2
X1
X2
Z1
Z2
0.2 0
0.8 0.1
V1
V2
Q1
K1
V1
Q2
K2
V2
<sos> Hallo
<sos> 0.2 0
Hello 0.8 0.1
Similarity Matrix
Masked Multi Head Attention
WQ
WK
WV
Positional Encoding
Embedding
<sos> Hallo